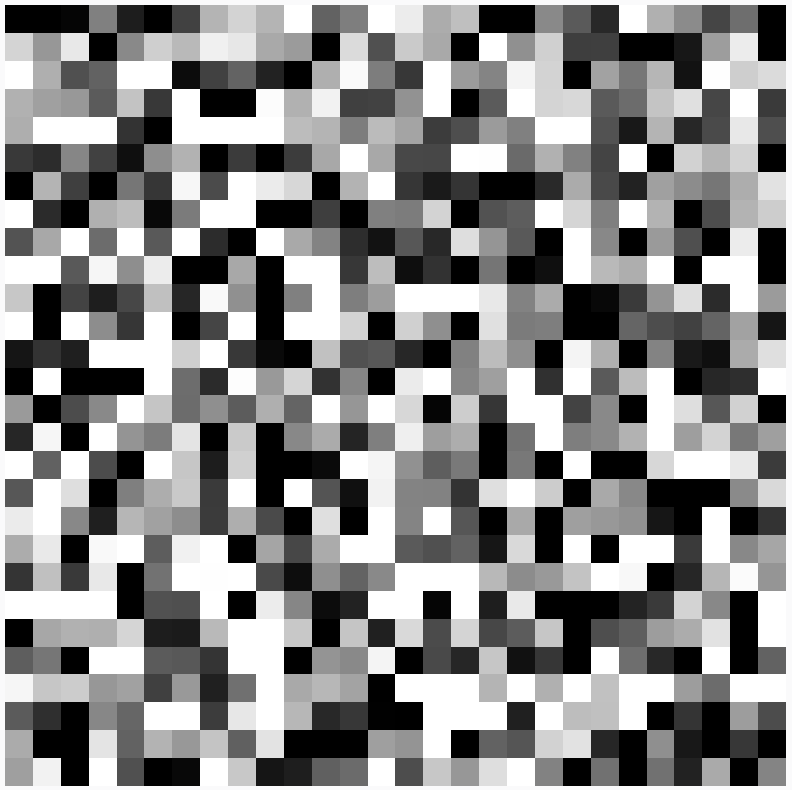
How could this become an image?

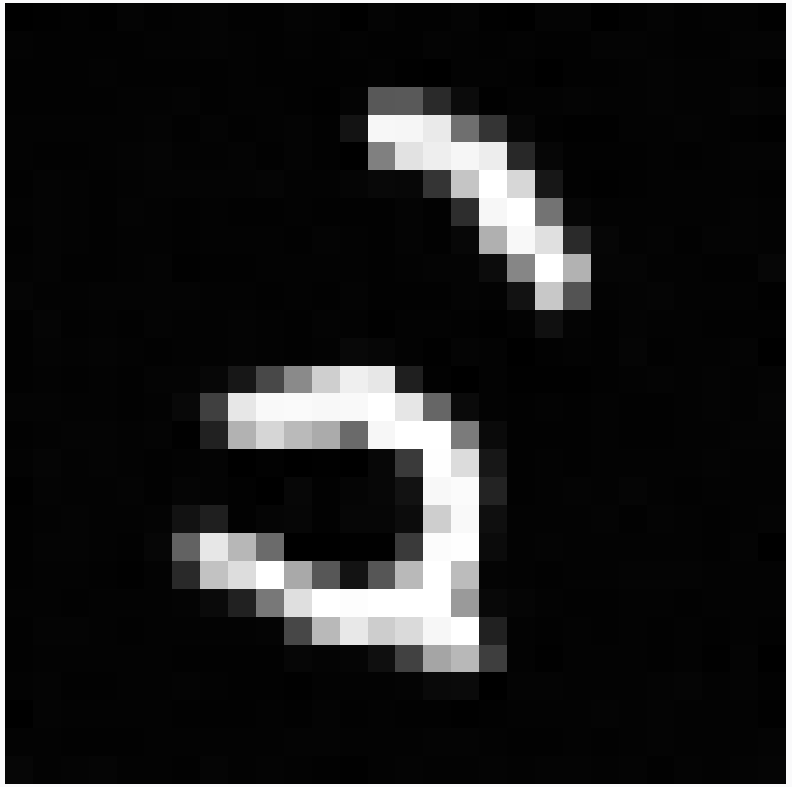


Measured replay starts from a fresh Gaussian draw

> What computation could connect these two states?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

How could this become an image?

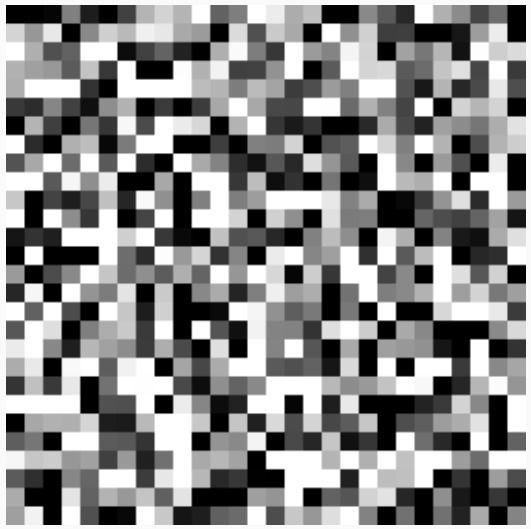


Same trajectory after 1,000 ancestral steps

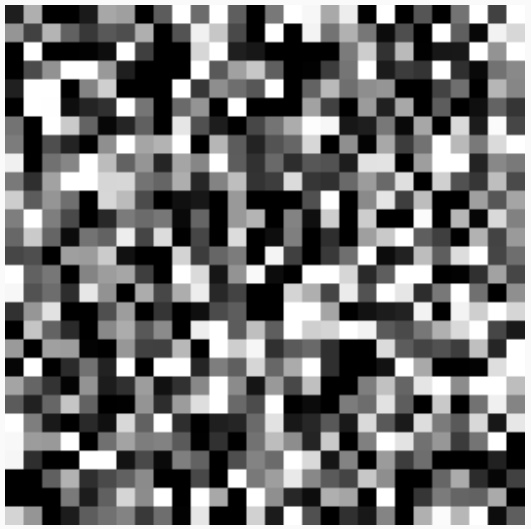
> A sequence of learned reverse transitions, starting from noise.

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

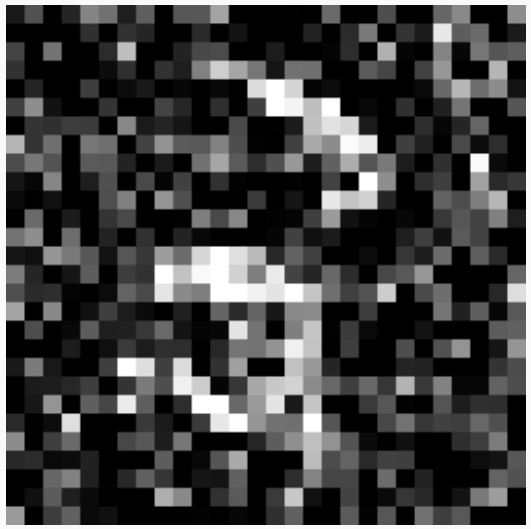
Many small decisions make one sample



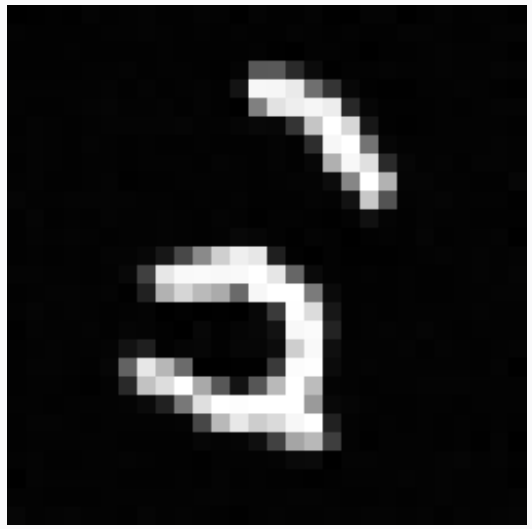
t = 1000



t = 600



t = 200

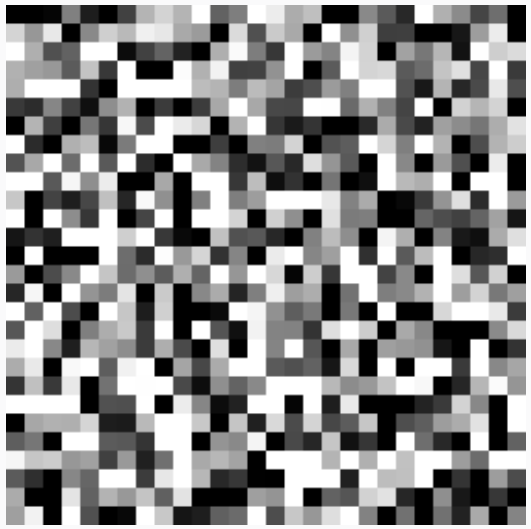


t = 0

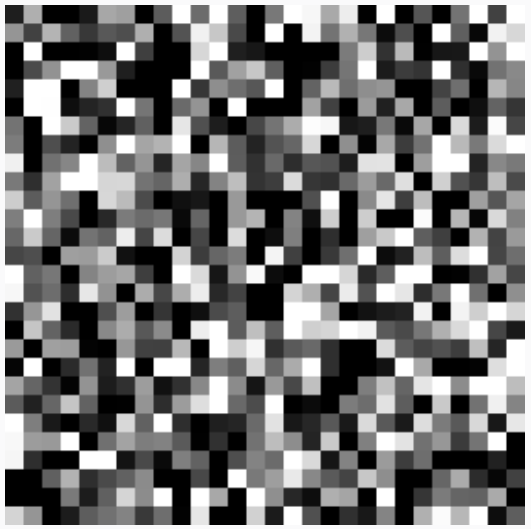
Which clean training image is this sampler trying to reconstruct?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

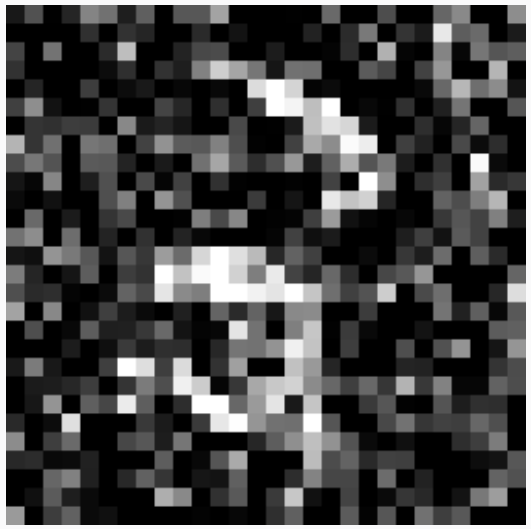
Many small decisions make one sample



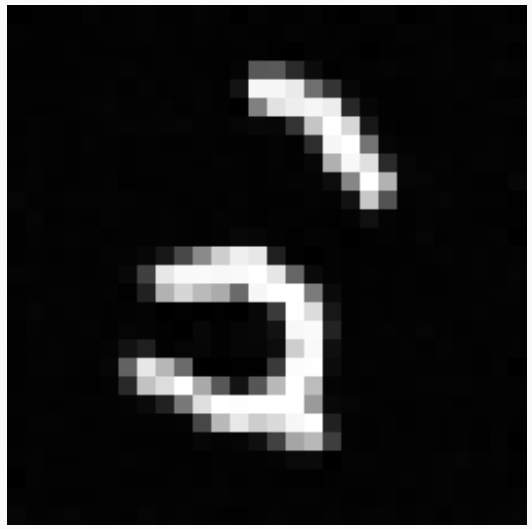
t = 1000



t = 600



t = 200

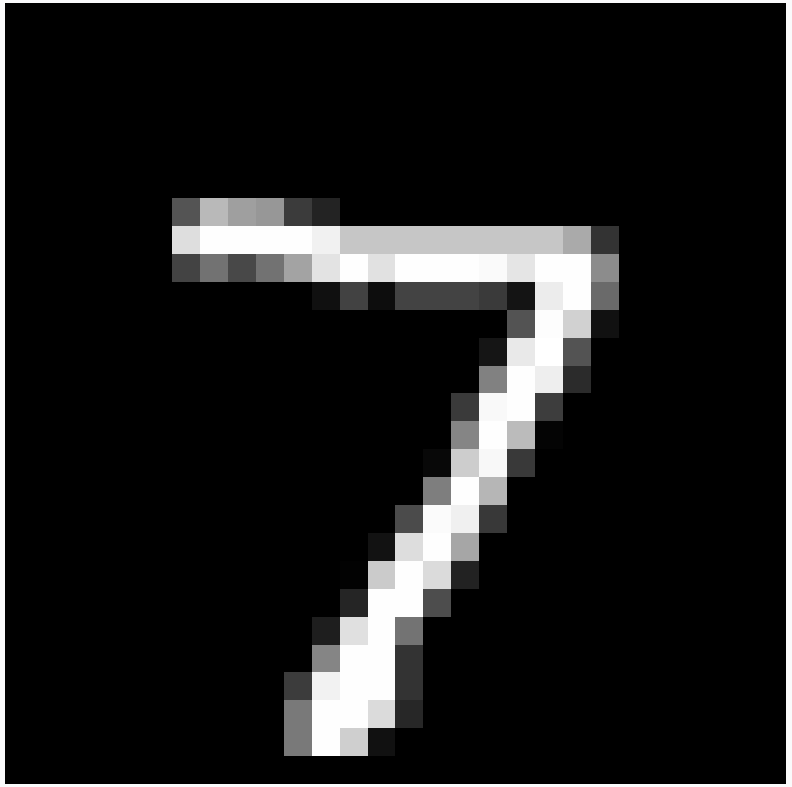


t = 0

Replayed 2,000-update teaching checkpoint; this is generated data, not recovery of a hidden original image.

None. It starts from new noise and generates a new sample.

For this lecture, follow one handwritten 7



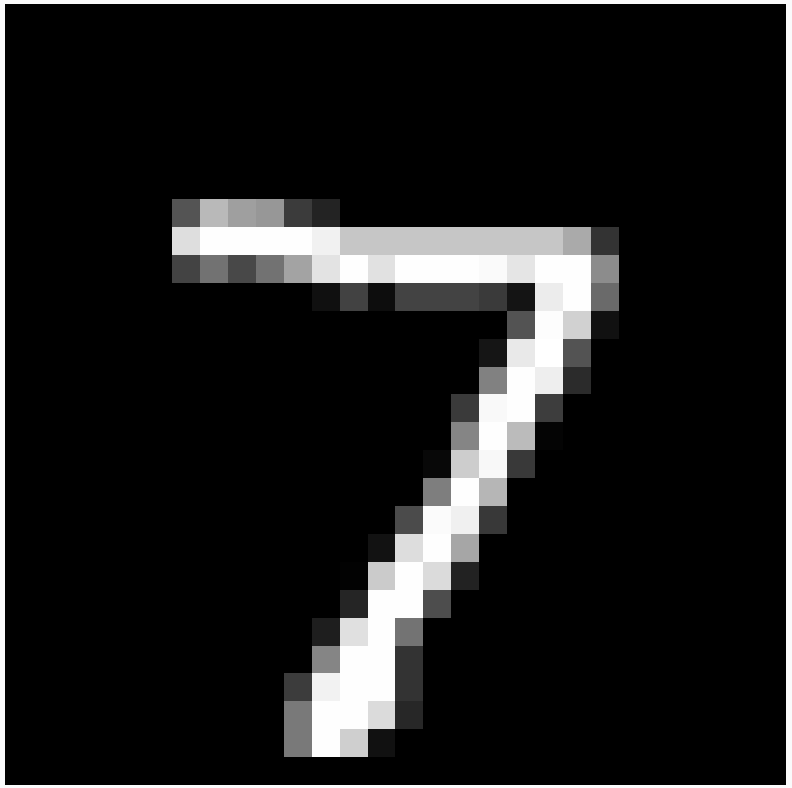
MNIST test index 0

28 × 28 greyscale

$x_0 = \text{uint8}/127.5 - 1$

> Why keep one image visible through the calculations?

For this lecture, follow one handwritten 7



MNIST test index 0

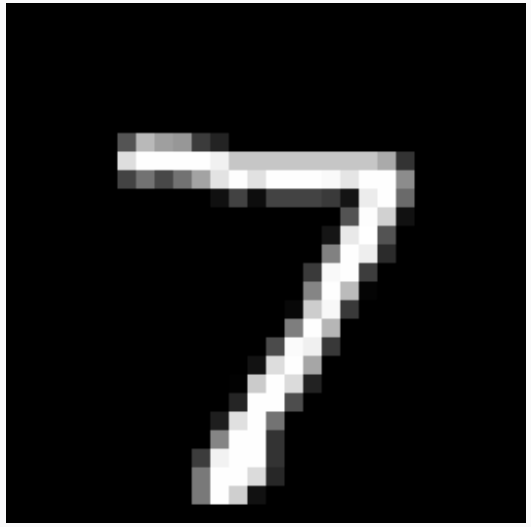
28 × 28 greyscale

$x_0 = \text{uint8}/127.5 - 1$

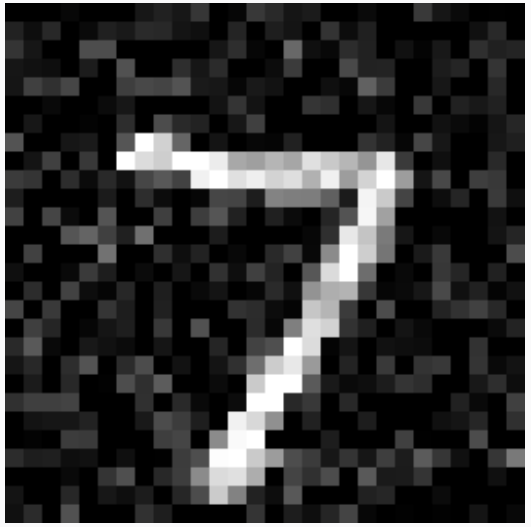
Earlier AE/VAE examples used the held 3. This 7 preserves the validated diffusion experiment.

> It lets us connect each symbol to a concrete state rather than swap examples silently.

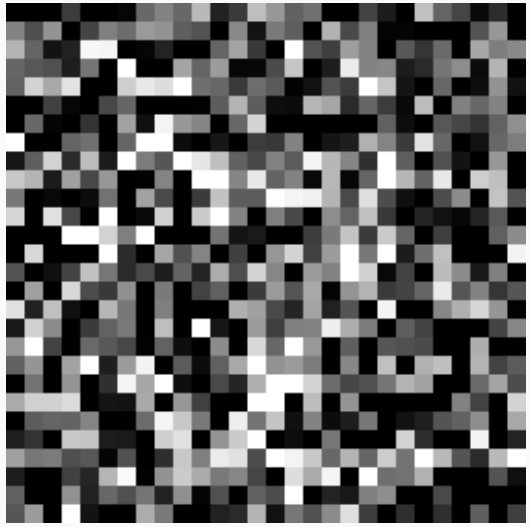
The easy direction destroys information



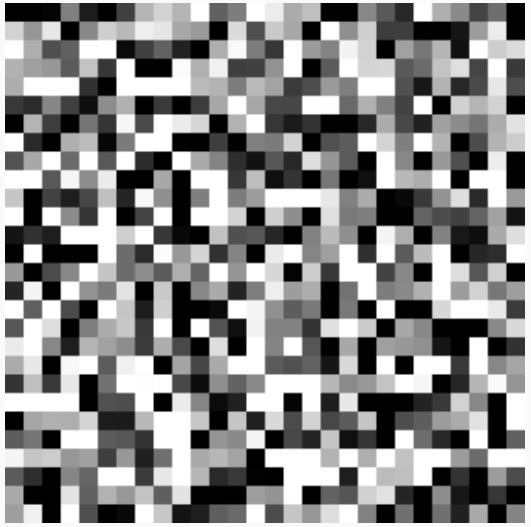
clean x_0



t = 100



t = 350

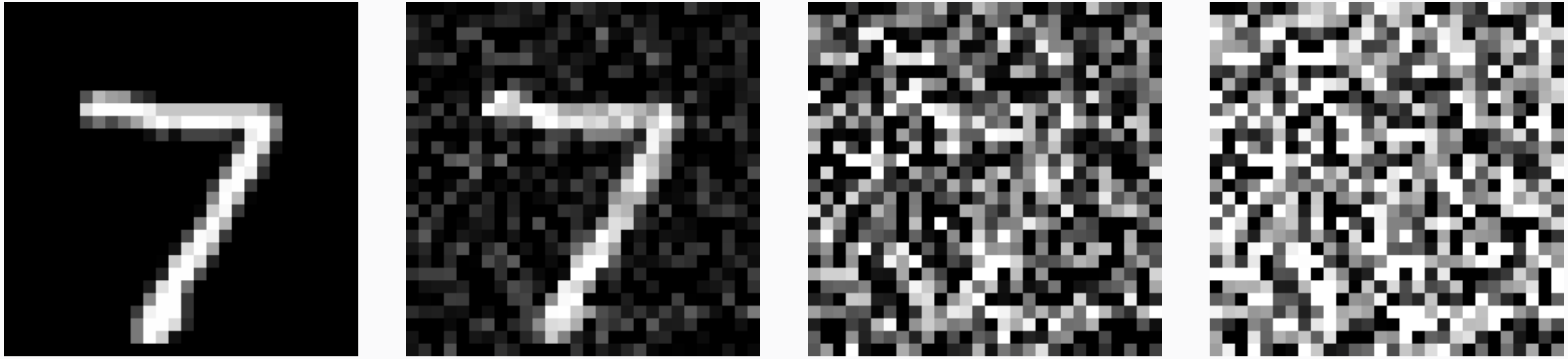


t = 1000

> Is this particular display a sequential Markov-chain path?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

The easy direction destroys information

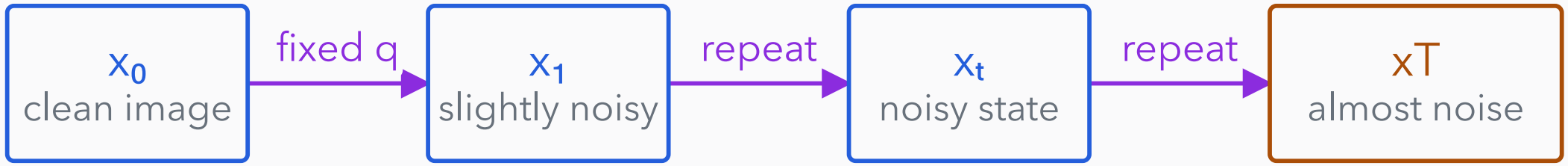


clean x_0 | t = 100 | t = 350 | t = 1000

Fixed clean image and noise draw for comparison; each panel is constructed directly from $q(x_t|x_0)$.

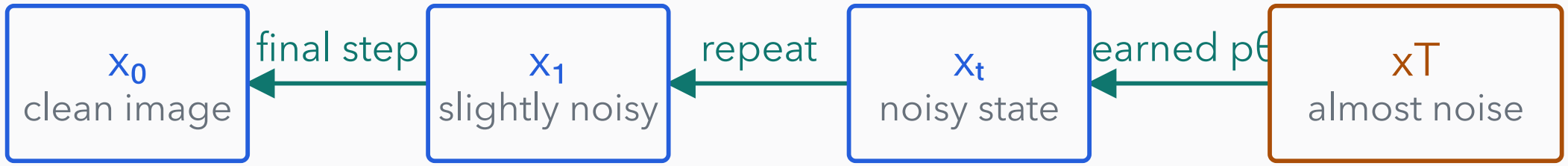
> No. It uses one fixed noise draw to compare direct marginals at different t.

Known corruption; learned reverse transitions



Which direction requires training?

Known corruption; learned reverse transitions



The reverse distribution. The forward corruption schedule is chosen and fixed.

One pixel, one small corruption

$$x_1 = \sqrt{1 - \beta_1}\, x_0 + \sqrt{\beta_1}\, \epsilon_1$$

The square roots scale standard deviations; β_1 is the added variance.

> Why not multiply noise by β_1 directly?

One pixel, one small corruption

$$x_1 = \sqrt{1-\beta_1}\, x_0 + \sqrt{\beta_1}\, \epsilon_1$$

$$\epsilon_1 \sim \mathcal{N}(0,1), \qquad \beta_1 = 0.0001$$

The square roots scale standard deviations; β_1 is the added variance.

> A standard-normal noise term multiplied by $\sqrt{\beta_1}$ has variance β_1.

The forward step is a Gaussian conditional

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\, x_{t-1},\, \beta_t I)$$

Second Gaussian argument is covariance. q here is the fixed corruption process.

> Does q contain a learned encoder in this lecture?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

The forward step is a Gaussian conditional

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\, x_{t-1},\, \beta_t I)$$

$$\alpha_t = 1 - \beta_t$$

Second Gaussian argument is covariance. q here is the fixed corruption process.

> No. The symbol q names a fixed, declared noising distribution.

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

One step and many steps need different symbols

symbol	meaning
β_t	noise variance added at step t
$\alpha_t = 1 - \beta_t$	signal variance retained in one step
$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$	cumulative signal variance retained

> Is $\bar{\alpha}_t$ the same as α_t?

One step and many steps need different symbols

symbol	meaning
β_t	noise variance added at step t
$\alpha_t = 1 - \beta_t$	signal variance retained in one step
$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$	cumulative signal variance retained

$$\bar{\alpha}_0 = 1$$

> Only at the first step. The bar denotes a product over the preceding steps.

Expand two corruption steps

$$x_2 = \sqrt{\alpha_2}\left(\sqrt{\alpha_1}x_0 + \sqrt{\beta_1}\epsilon_1\right) + \sqrt{\beta_2}\epsilon_2$$

> Why can we combine the two noise terms?

Expand two corruption steps

$$x_2 = \sqrt{\alpha_2}\left(\sqrt{\alpha_1}x_0 + \sqrt{\beta_1}\epsilon_1\right) + \sqrt{\beta_2}\epsilon_2$$

$$\mathrm{Var}(\text{noise}) = \alpha_2\beta_1 + \beta_2 = 1 - \alpha_1\alpha_2$$

Independent Gaussian terms sum to a Gaussian; their variances add after scaling.

Jump directly to any chosen timestep

$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\, x_0, (1 - \bar{\alpha}_t) I\right)$$

> Must training simulate all earlier corruption steps?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Jump directly to any chosen timestep

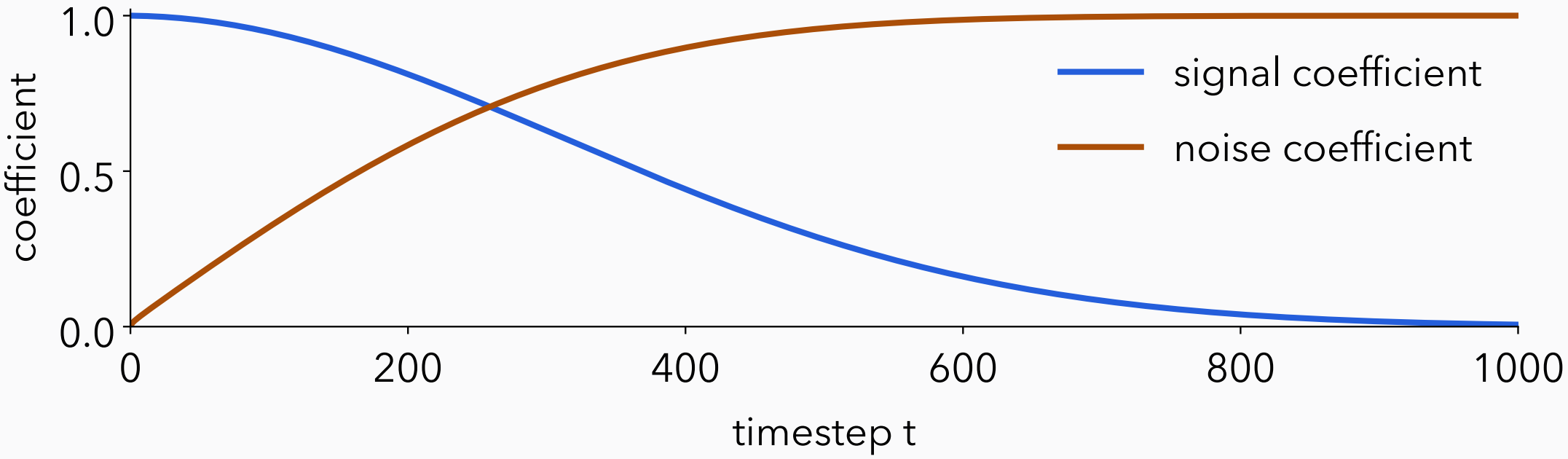
$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\, x_0, (1 - \bar{\alpha}_t) I\right)$$

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

> No. This direct construction draws the correct marginal at the selected t.

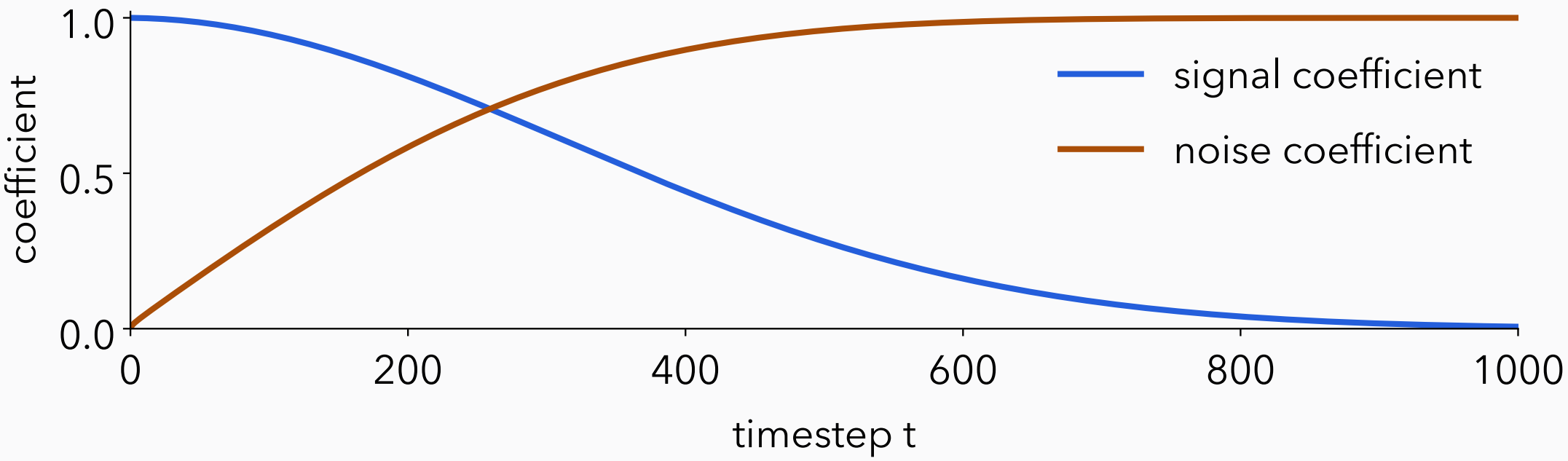
Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Choose a schedule whose endpoint is near noise



Why is setting T = 20 with the same endpoints a different model?

Choose a schedule whose endpoint is near noise



Linear β_t from 0.0001 to 0.02, T = 1000. $\bar{\alpha}_{1000} \approx 0.00004036$.

> It changes the accumulated product and may leave substantial signal at the endpoint.

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

The bar carries the cumulative effect

t	β_t	$\bar{\alpha}_t$
1	0.00010	0.999900
100	0.00207	0.897018
350	0.00705	0.285200
1000	0.02000	0.000040

Which coefficient belongs in the direct x_t formula?

The bar carries the cumulative effect

t	β_t	$\bar{\alpha}_t$
1	0.00010	0.999900
100	0.00207	0.897018
350	0.00705	0.285200
1000	0.02000	0.000040

One-based indexing in maths and code. $\bar{\alpha}[0] = 1$ is an explicit boundary value.

> $\bar{\alpha}_t$, not α_t.

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Choose one scalar training example

quantity	chosen value
clean pixel x_0	0.8
timestep t	350
noise ε	−0.4

> Which of these three quantities is learned?

Choose one scalar training example

quantity	chosen value
clean pixel x_0	0.8
timestep t	350
noise ε	−0.4

Chosen numerical illustration. The following image example uses actual MNIST pixels.

> None. They define one training example; model weights are learned later.

Compute the noisy pixel

$$x_{350} = \sqrt{\bar{\alpha}_{350}}(0.8) + \sqrt{1 - \bar{\alpha}_{350}}(-0.4)$$

> Is the result required to stay in $[-1, 1]$?

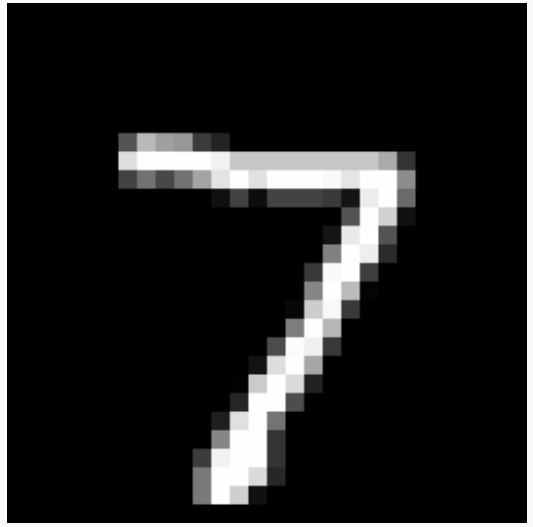
Compute the noisy pixel

$$x_{350} = \sqrt{\bar{\alpha}_{350}}(0.8) + \sqrt{1 - \bar{\alpha}_{350}}(-0.4)$$

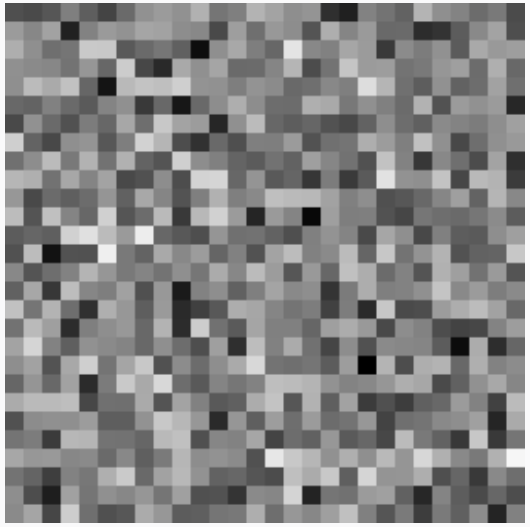
$$x_{350} \approx 0.089049$$

> No. Gaussian corruption has unbounded support. Do not clip training states.

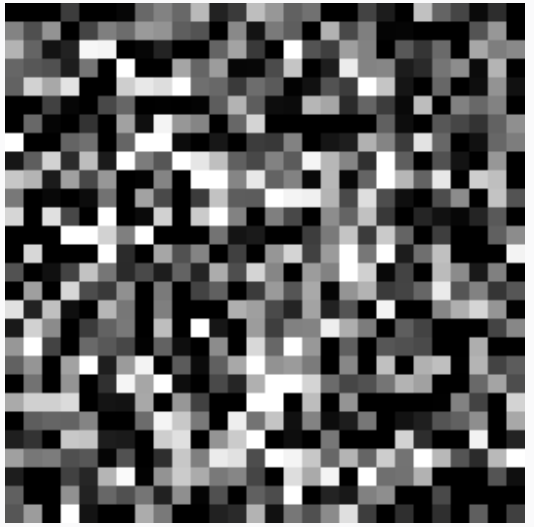
Apply the same construction to every pixel



x_0 · real image



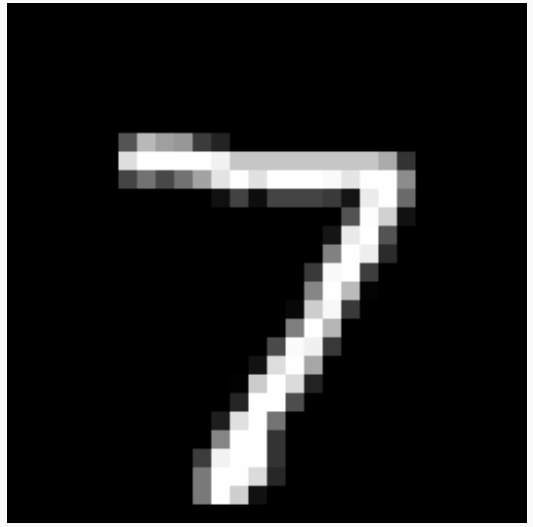
ε · known draw



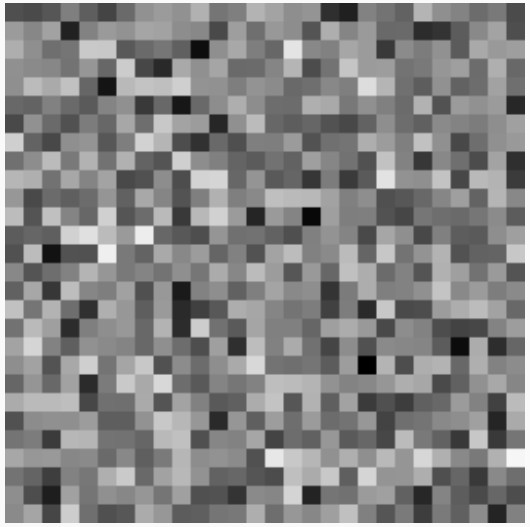
x_{350} · constructed input

> Are noise values shared across pixels?

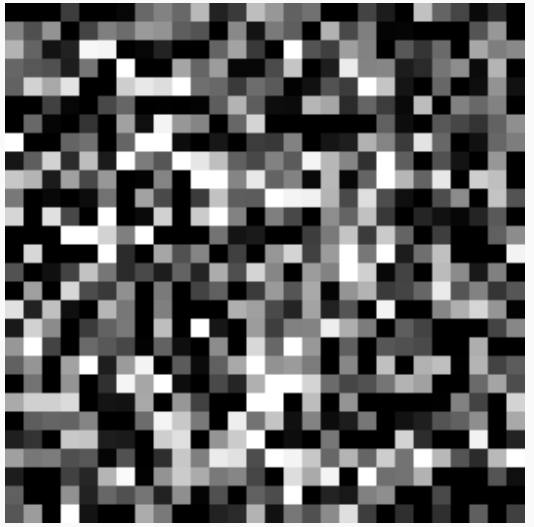
Apply the same construction to every pixel



x_0 · real image



ε · known draw

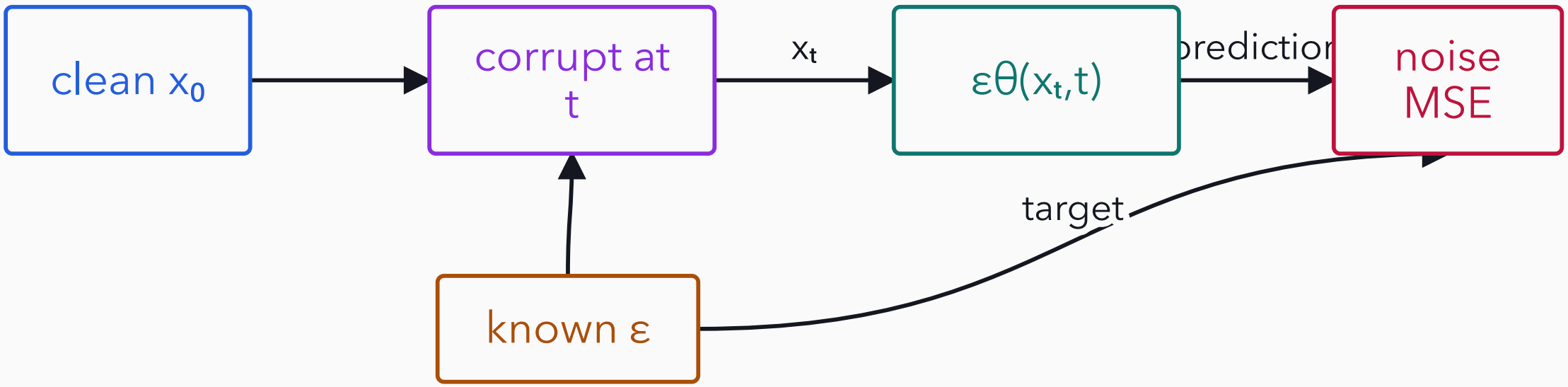


x_{350} · constructed input

$$\text{all three tensors have shape } 1 \times 1 \times 28 \times 28$$

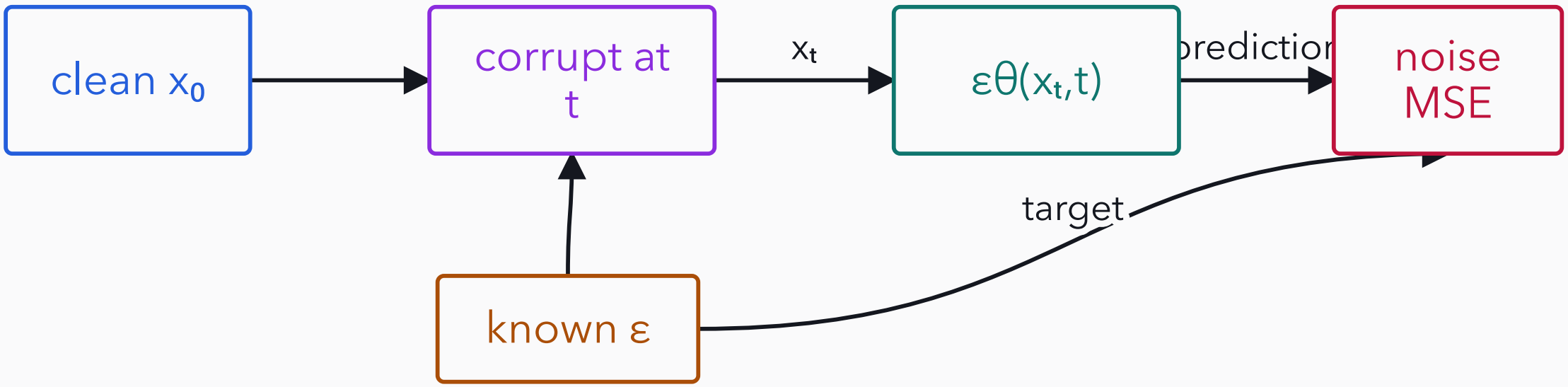
> No. Each pixel receives a Gaussian draw; the same scalar coefficients apply to the full tensor.

What does one training example contain?



> Does the denoiser see x_0 during its forward call?

What does one training example contain?



Input: x_t and t. Target: the injected ε. The clean image helps construct the input, not the network's forward pass.

> No. It receives x_t and t; ε is supplied to the loss as the target.

Training knows how the input was made

quantity	available at training?	network input?
x_0	yes	no
t	yes	yes
ε	yes	no; target
x_t	yes	yes

> Why can this be supervised without manual noise labels?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Training knows how the input was made

quantity	available at training?	network input?
x_0	yes	no
t	yes	yes
ε	yes	no; target
x_t	yes	yes

The target is known because we generated the corruption ourselves.

> We sampled ε and used it to construct x_t, so the label is already known.

The same value can mean different things at different t

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

> Could a denoiser ignore the noise level?

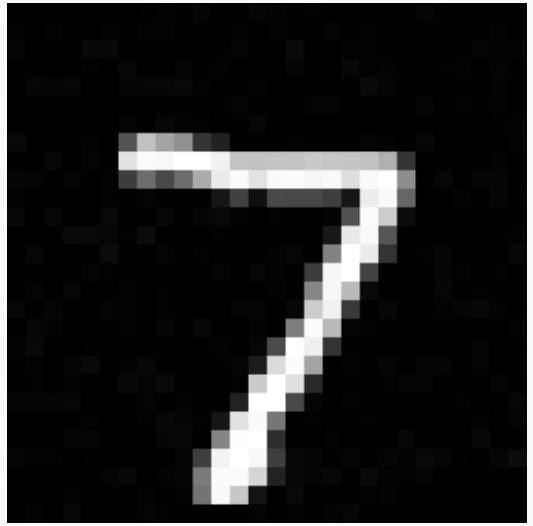
The same value can mean different things at different t

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

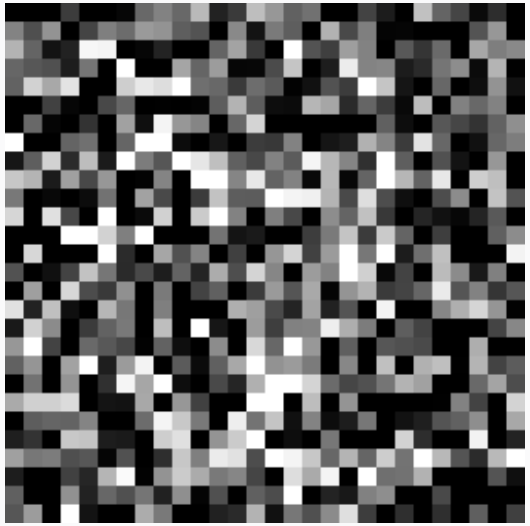
A faint structure at low noise and the same structure at high noise imply different uncertainty.

> It would mix prediction tasks with different signal/noise proportions. t identifies the task.

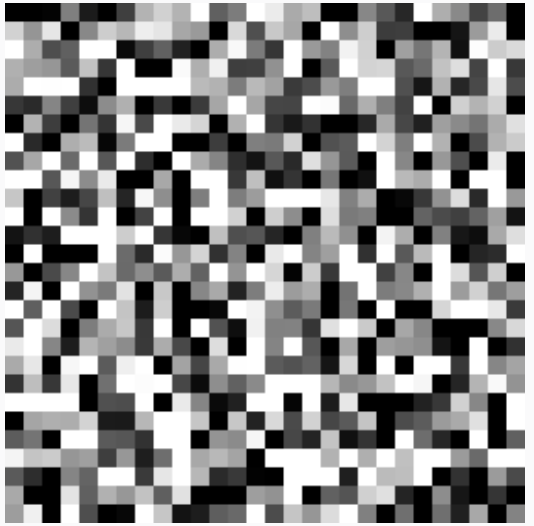
The prediction problem changes with noise level



t = 7



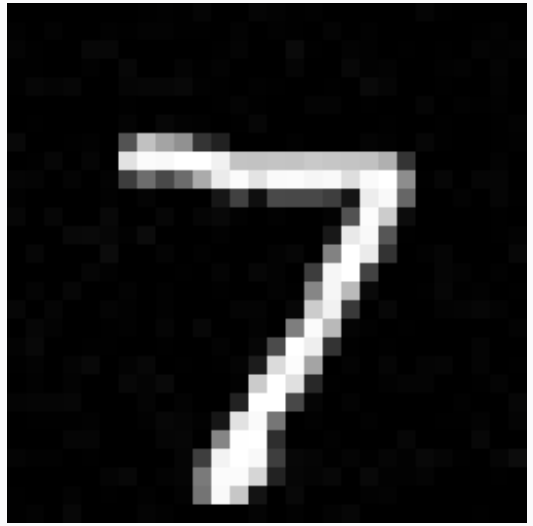
t = 350



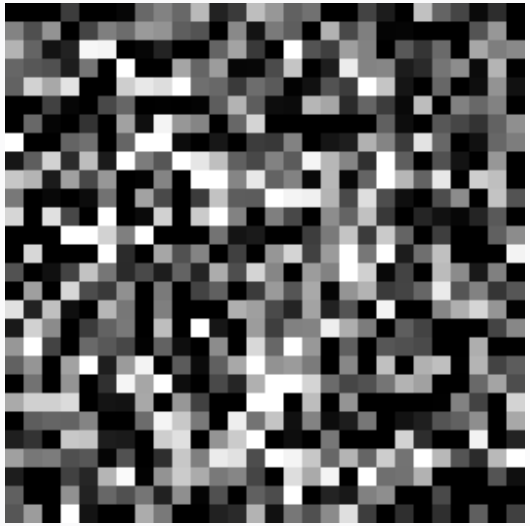
t = 920

At which stage is the original image least identifiable?

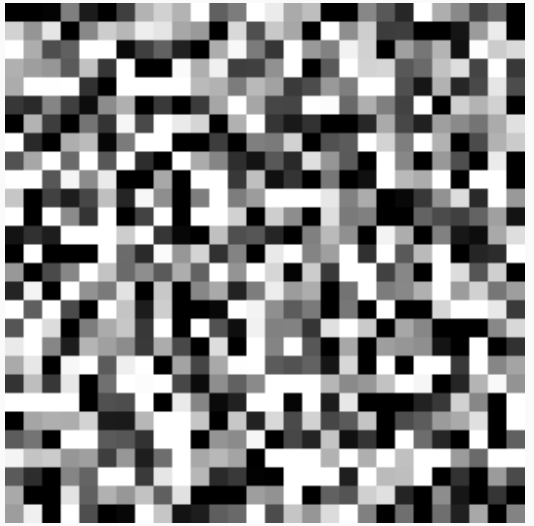
The prediction problem changes with noise level



t = 7



t = 350



t = 920

Same held image and base noise for this comparison. Training normally resamples both t and ε.

> At high t, where the remaining signal coefficient is very small.

One noisy input can have several explanations

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

> Can the network always recover the exact injected ε from x_t alone?

One noisy input can have several explanations

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

Different clean images and noise draws can lead to the same noisy value.

> No. The inverse problem is ambiguous; training minimises an expected prediction error.

Squared error learns a conditional average

$$\epsilon_\theta(x_t, t) \;\approx\; \mathbb{E}[\epsilon \mid x_t, t]$$

> Is an imperfect match to a particular ε necessarily a bug?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Squared error learns a conditional average

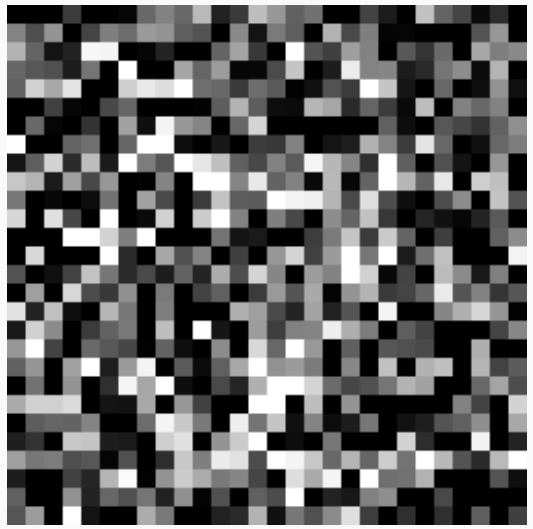
$$\epsilon_\theta(x_t, t) \;\approx\; \mathbb{E}[\epsilon \mid x_t, t]$$

Population squared-error optimum with sufficient capacity and successful optimisation. A finite network only approximates it.

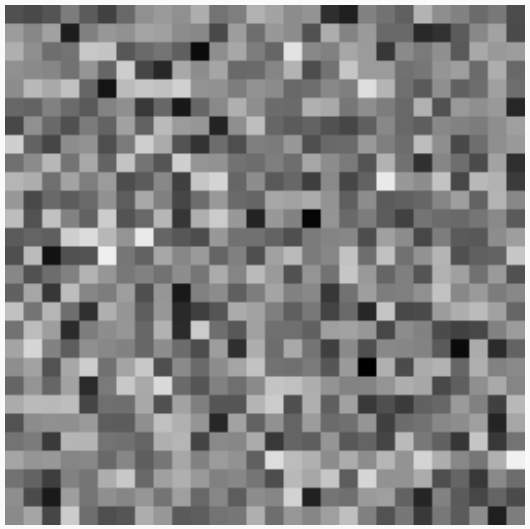
> No. Some uncertainty is irreducible from the available input.

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

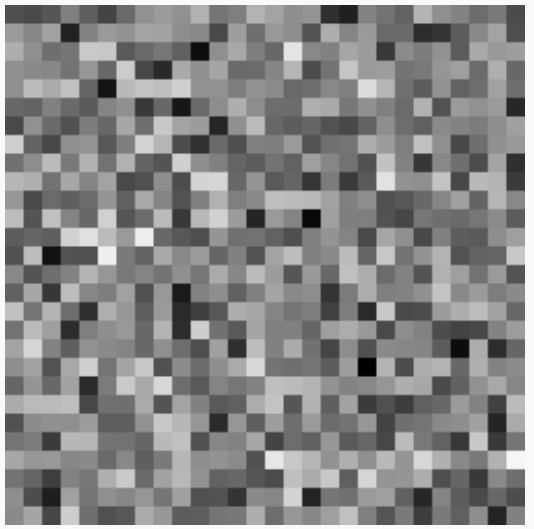
The network predicts a noise tensor



input x_{350}



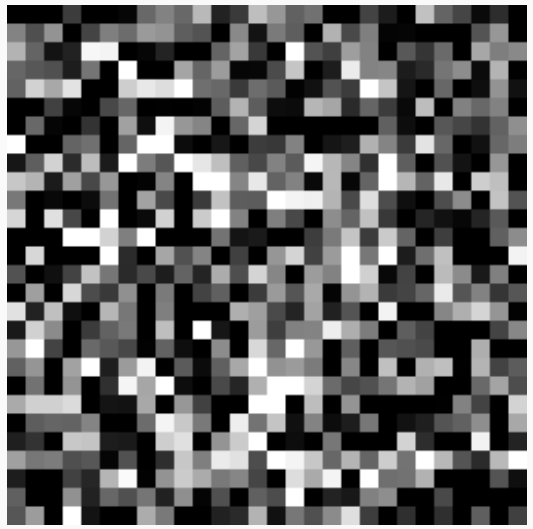
predicted $\varepsilon\theta$



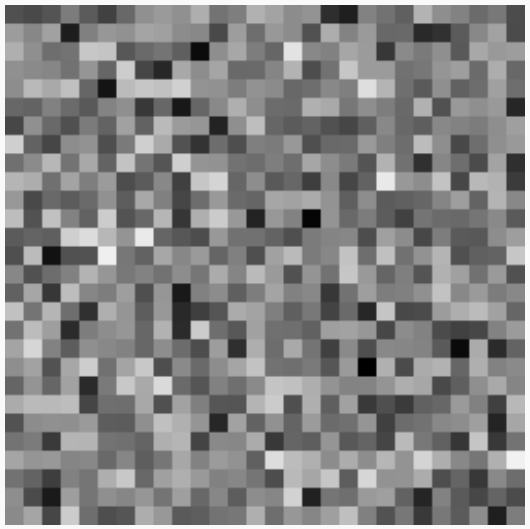
target ε

> Why does the network output look like noise?

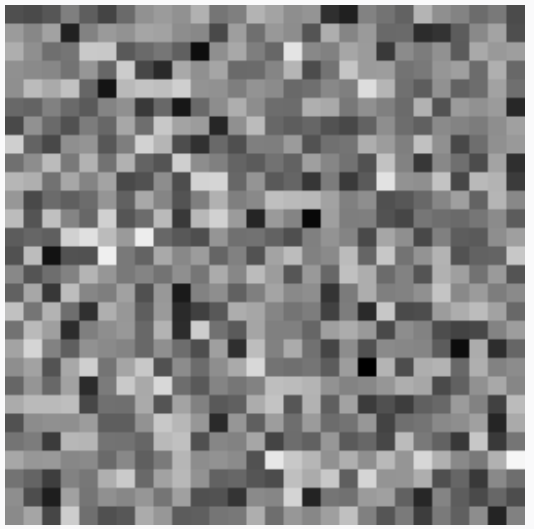
The network predicts a noise tensor



input x_{350}



predicted $\varepsilon\theta$



target ε

Measured checkpoint prediction. Noise panels use a display range of $[-3,3]$; the image uses $[-1,1]$.

> That is the declared target. A clean estimate is computed from it in a later step.

The output shape matches the noise target

tensor	shape
clean/noisy images	B × 1 × 28 × 28
timesteps	B integer values
predicted noise	B × 1 × 28 × 28
target noise	B × 1 × 28 × 28

> What shape mistake could silently corrupt the construction?

The output shape matches the noise target

tensor	shape
clean/noisy images	B × 1 × 28 × 28
timesteps	B integer values
predicted noise	B × 1 × 28 × 28
target noise	B × 1 × 28 × 28

Broadcast $\bar{\alpha}_t$ as B × 1 × 1 × 1, not along a spatial axis.

> Mixing batch coefficients with channel or spatial positions. Assert tensor shapes.

Construct inputs and labels in one short block

```
t = torch.randint(1, T+1, (B,))
eps = torch.randn_like(x0)
a = alpha_bar[t].view(B, 1, 1, 1)
xt = a.sqrt()*x0 + (1-a).sqrt()*eps
pred = model(xt, t)
```

Where is the label tensor created?

Construct inputs and labels in one short block

```
t = torch.randint(1, T+1, (B,))
eps = torch.randn_like(x0)
a = alpha_bar[t].view(B, 1, 1, 1)
xt = a.sqrt()*x0 + (1-a).sqrt()*eps
pred = model(xt, t)
```

Sample x_0 from data, t uniformly, and ε independently.

eps is the label as well as the known random input to corruption.

Begin with a predictor we can differentiate by hand

$$\hat{\epsilon}_\theta = \theta x_t, \qquad \theta = 0.3$$

Scalar teaching predictor, not the MNIST denoiser architecture.

> What is the single trainable quantity?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Begin with a predictor we can differentiate by hand

$$\hat{\epsilon}_\theta = \theta x_t, \qquad \theta = 0.3$$

$$\hat{\epsilon}_\theta \approx 0.026715$$

Scalar teaching predictor, not the MNIST denoiser architecture.

> θ. The selected x_0, t and ε remain fixed for this inspected update.

Compare prediction with the injected noise

$$e = \hat{\epsilon}_\theta - \epsilon$$

> Which tensor is the target in this subtraction?

Compare prediction with the injected noise

$$e = \hat{\epsilon}_\theta - \epsilon$$

$$e \approx 0.026715 - (-0.4) \approx 0.426715$$

> The sampled ε, not x_0 or x_t.

A squared residual gives one scalar loss

$$L = (\theta x_t - \epsilon)^2$$

> Is this already the batch mean?

A squared residual gives one scalar loss

$$L = (\theta x_t - \epsilon)^2$$

$$L \approx 0.182085$$

> It is one scalar example. Image and batch reductions come next.

Differentiate through the prediction

$$\frac{\partial L}{\partial \theta} = 2(\theta x_t - \epsilon)x_t$$

Autodiff and central finite differences agree within 10^{-9}.

> Why hold ε fixed during a finite-difference check?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Differentiate through the prediction

$$\frac{\partial L}{\partial \theta} = 2(\theta x_t - \epsilon)x_t$$

$$\frac{\partial L}{\partial \theta} \approx 0.075997$$

Autodiff and central finite differences agree within 10^{-9}.

> Otherwise changes in the random draw contaminate the comparison between perturbed parameters.

One update, then recompute the same loss

quantity	before	after η = 0.1
θ	0.300000	0.292400
same-example loss	0.182085	0.181508

> Does this same-example decrease prove a good image model?

One update, then recompute the same loss

quantity	before	after η = 0.1
θ	0.300000	0.292400
same-example loss	0.182085	0.181508

$$\theta^{+} = \theta - 0.1\,\partial L/\partial\theta$$

> No. It verifies the direction and arithmetic of a local update.

State exactly what “mean squared error” averages

$$L_{\text{simple}} = \frac{1}{BCHW} \sum_{i,c,h,w} \left(\epsilon_{i,c,h,w} - \epsilon_\theta(x_{t_i}, t_i)_{c,h,w}\right)^2$$

> How does an event sum followed by batch mean differ?

State exactly what “mean squared error” averages

$$L_{\text{simple}} = \frac{1}{BCHW} \sum_{i,c,h,w} \left(\epsilon_{i,c,h,w} - \epsilon_\theta(x_{t_i}, t_i)_{c,h,w}\right)^2$$

Here C = 1, H = W = 28. PyTorch mse_loss defaults to this mean over all elements.

> For fixed image size it differs by the factor CHW, changing the scale and effective learning rate.

A complete training update is short

```
pred = model(xt, t)
loss = F.mse_loss(pred, eps)
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

Which variables does the optimiser update?

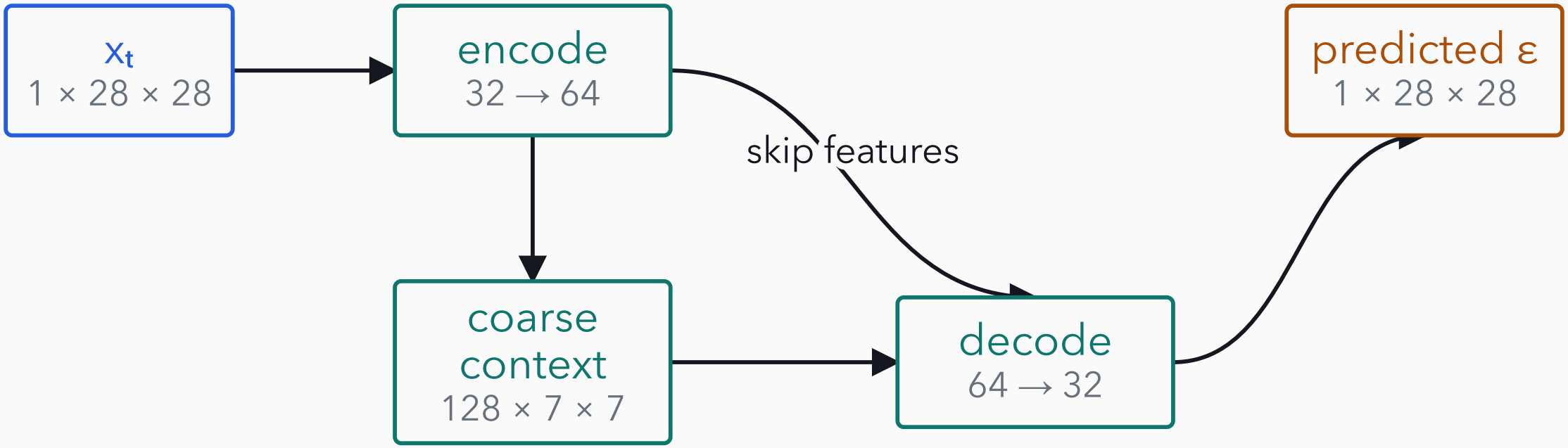
A complete training update is short

```
pred = model(xt, t)
loss = F.mse_loss(pred, eps)
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

The network sees a new corruption on later training iterations.

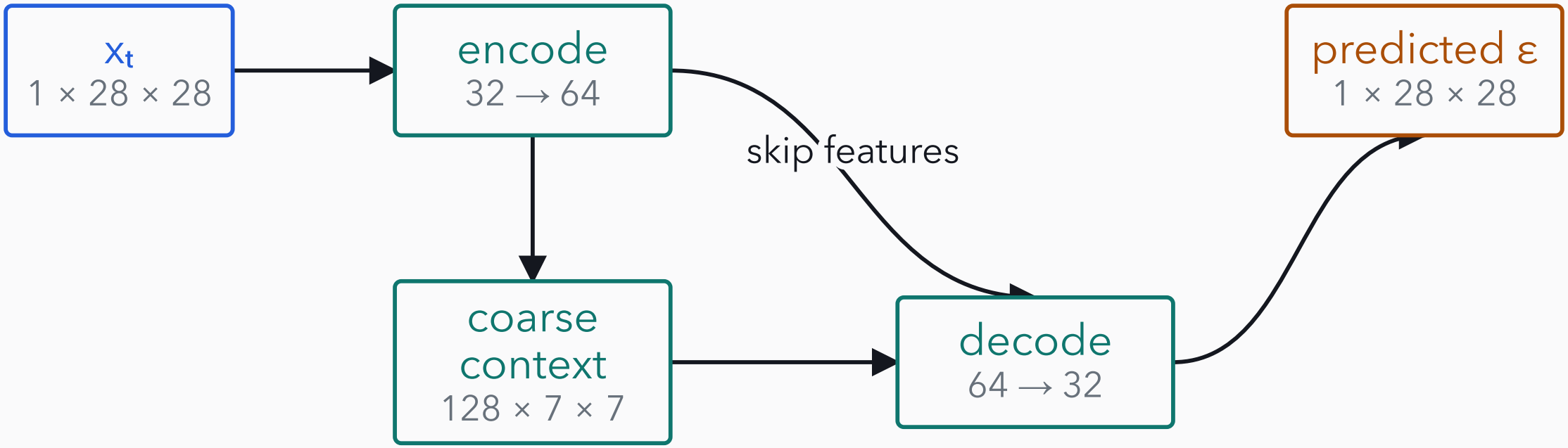
> Only the denoiser parameters θ. The forward schedule stays fixed.

A small U-Net keeps context and detail



Why retain high-resolution features?

A small U-Net keeps context and detail



Schematic of the saved 814,145-parameter model; full layers are in ddpm.py.

> They help localise details while the coarse path supplies wider spatial context.

The timestep enters the network explicitly



> Do we train a separate network at each timestep?

The timestep enters the network explicitly



A learned linear projection of the time embedding is added in each residual block.

> No. One set of weights is conditioned on t.

Inspect a real image-model update

quantity	fresh measured value
training batch	indices 0–3; t = 7,100,350,920
loss before	0.098449
loss after same batch	0.096120
gradient norm	1.625549

> Did we train on the held image shown earlier?

Inspect a real image-model update

quantity	fresh measured value
training batch	indices 0–3; t = 7,100,350,920
loss before	0.098449
loss after same batch	0.096120
gradient norm	1.625549

SGD η = 0.001 on a copy of the checkpoint. Test index 0 is reserved for inspection.

> No. This update uses the official training split.

Training and sampling start with different information

training	sampling
clean x_0 is available	no clean target image
sample t and ε	start at xT ~ N(0,I)
compare predicted ε to known ε	use predicted ε in a reverse step

> Can the sampler subtract the true injected noise?

Training and sampling start with different information

training	sampling
clean x_0 is available	no clean target image
sample t and ε	start at xT ~ N(0,I)
compare predicted ε to known ε	use predicted ε in a reverse step

The denoiser is shared; the surrounding computation is different.

> No. A newly generated state has no known training corruption label.

Solve the forward equation for a clean estimate

$$\widehat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

> Is this already the next noisy state x_{t-1}?

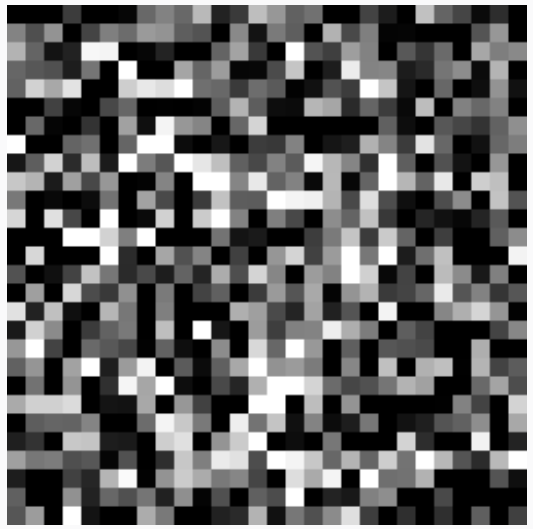
Solve the forward equation for a clean estimate

$$\widehat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

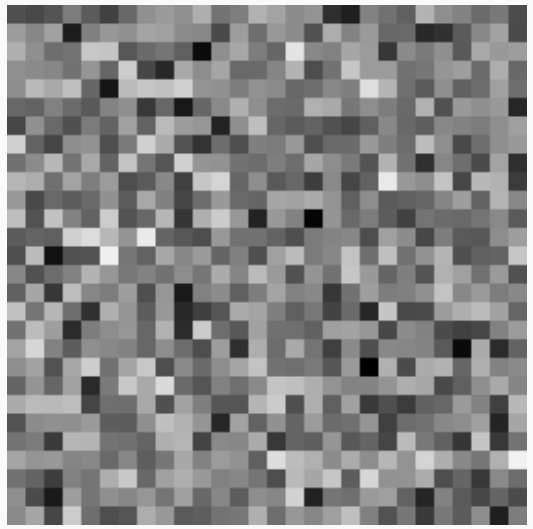
This is a clean-image estimate derived from the model's noise prediction.

No. It estimates x_0, while the sampler needs a distribution for the previous noisy state.

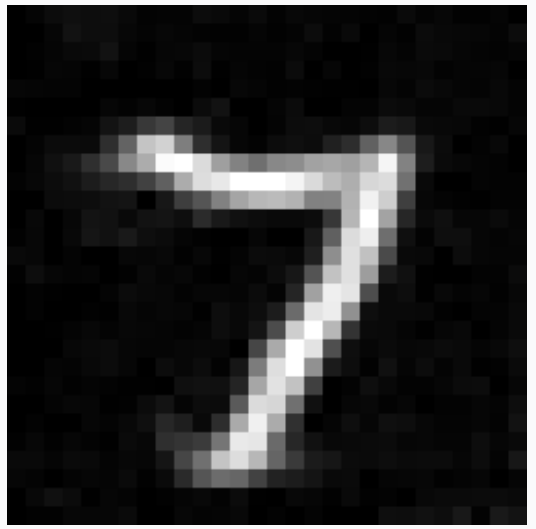
A clean estimate is useful, but is not a reverse step



current x_{350}



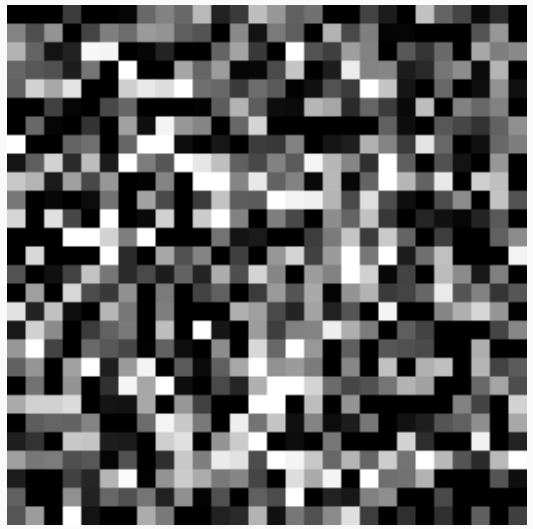
predicted $\varepsilon\theta$



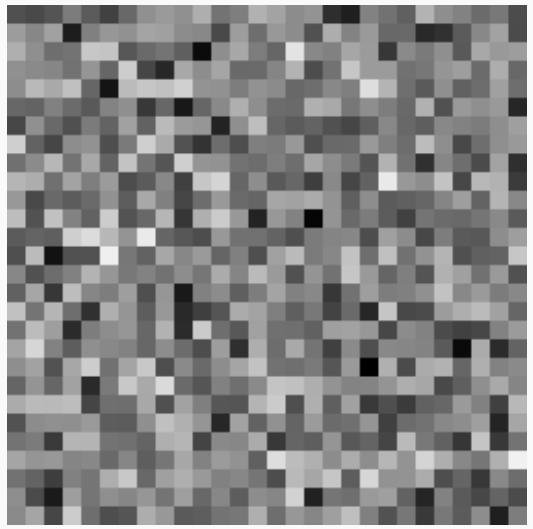
derived $\hat{x}_0$

> Would jumping straight to this estimate reproduce the DDPM sampler?

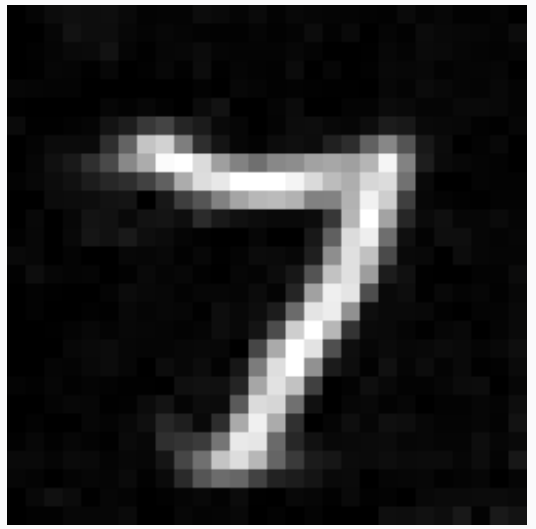
A clean estimate is useful, but is not a reverse step



current x_{350}



predicted $\varepsilon\theta$



derived $\hat{x}_0$

Measured checkpoint output. Display clipping is for these pictures only.

> No. The declared sampler takes successive reverse transitions.

Model the previous state with a Gaussian

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\left(\mu_\theta(x_t, t), \sigma_t^2 I\right), \quad t > 1$$

> Which part of this distribution is learned here?

Model the previous state with a Gaussian

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\left(\mu_\theta(x_t, t), \sigma_t^2 I\right), \quad t > 1$$

The mean depends on the denoiser. At t = 1, this sampler uses a deterministic mean readout.

> The mean through εθ. The variance is an explicit, fixed choice.

The predicted noise determines the reverse mean

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left[x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right]$$

Why are both α_t and $\bar{\alpha}_t$ present?

The predicted noise determines the reverse mean

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left[x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right]$$

Notice α_t outside and $\bar{\alpha}_t$ inside. They have different roles.

> One describes the local transition; the other describes cumulative corruption relative to x_0.

One scalar reverse step has explicit inputs

quantity	value
t	350
current x_t	0.089049
predicted $\varepsilon\theta$	0.25
fresh reverse noise ξ	−0.7

> Do we know the clean image during this calculation?

One scalar reverse step has explicit inputs

quantity	value
t	350
current x_t	0.089049
predicted $\varepsilon\theta$	0.25
fresh reverse noise ξ	−0.7

Chosen illustration. ξ is a new draw for sampling, distinct from the training corruption ε.

> The reverse formula uses x_t, t and the predicted noise, not a known x_0.

The clean estimate and reverse mean are different

computed object	value
clean estimate $\hat{x}_0$	-0.229038
reverse mean $\mu\theta$	0.087272
current state x_t	0.089049

> Why is the previous-state mean closer to x_t than $\hat{x}_0$ is?

The clean estimate and reverse mean are different

computed object	value
clean estimate $\hat{x}_0$	-0.229038
reverse mean $\mu\theta$	0.087272
current state x_t	0.089049

The reverse mean also agrees with the posterior-mean expression using $\hat{x}_0$; residual $< 10^{-12}$.

> It is one small reverse transition, not the endpoint of the full denoising chain.

Choose the reverse variance explicitly

$$\sigma_t^2 = \tilde{\beta}_t = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}$$

Posterior-variance convention for this sampler; it is not an assertion that the true marginal reverse conditional is known.

> Why does the t = 1 boundary need special attention?

Choose the reverse variance explicitly

$$\sigma_t^2 = \tilde{\beta}_t = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}$$

$$\tilde{\beta}_{350} \approx 0.0070321$$

Posterior-variance convention for this sampler; it is not an assertion that the true marginal reverse conditional is known.

> $\bar{\alpha}_0 = 1$ makes this added variance zero at the last step.

Draw around the reverse mean

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t \xi, \qquad \xi \sim \mathcal{N}(0, I)$$

> If ξ were positive instead, would this be the same sample?

Draw around the reverse mean

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t \xi, \qquad \xi \sim \mathcal{N}(0, I)$$

$$x_{349} \approx 0.087272 + 0.083857(-0.7) \approx 0.028572$$

> No. It would be another draw from the specified reverse Gaussian.

At the final step, do not add noise

$$x_0 = \mu_\theta(x_1, 1)$$

> What happens if we add ordinary noise again at t = 1?

At the final step, do not add noise

$$x_0 = \mu_\theta(x_1, 1)$$

```
mean = reverse_mean(x, pred, t)
if t > 1:
    x = mean + variance[t].sqrt()*noise
else:
    x = mean
```

We corrupt the final sample instead of following the declared sampler.

The complete sampling loop uses the model repeatedly

```
x = torch.randn(shape)
for t in range(T, 0, -1):
    pred = model(x, time_batch(t))
    noise = randn_like(x) if t > 1 else zeros_like(x)
    x = reverse_step(x, pred, t, schedule, noise)
```

Where does the sampler obtain its first state?

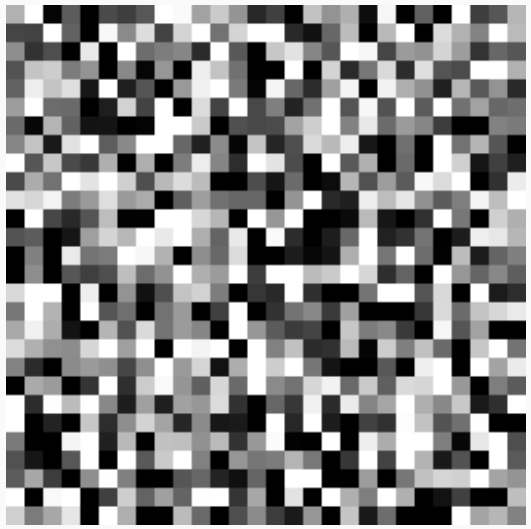
The complete sampling loop uses the model repeatedly

```
x = torch.randn(shape)
for t in range(T, 0, -1):
    pred = model(x, time_batch(t))
    noise = randn_like(x) if t > 1 else zeros_like(x)
    x = reverse_step(x, pred, t, schedule, noise)
```

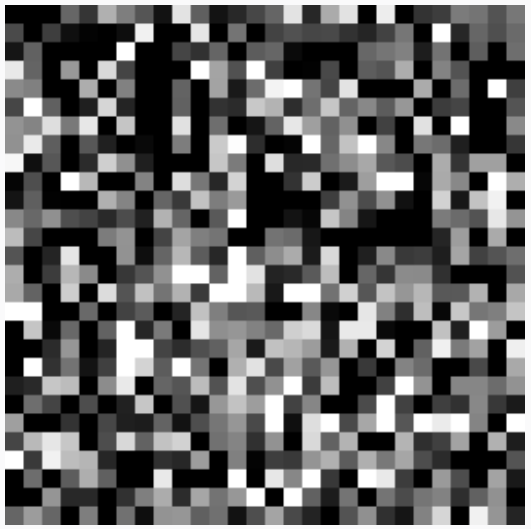
No gradient update during sampling. Keep network weights fixed and use evaluation mode.

A new standard-normal tensor, justified by the near-noise endpoint of the forward schedule.

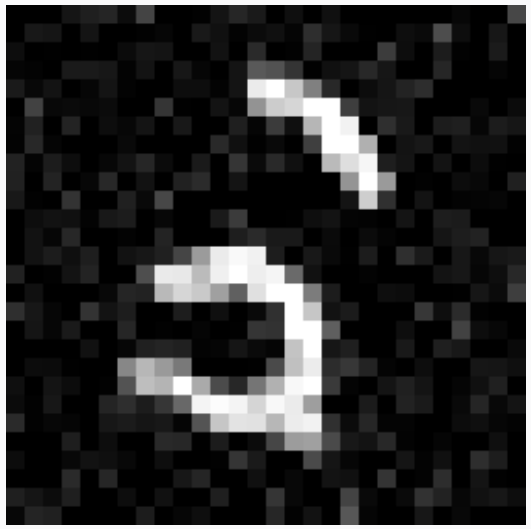
Inspect successive states on one trajectory



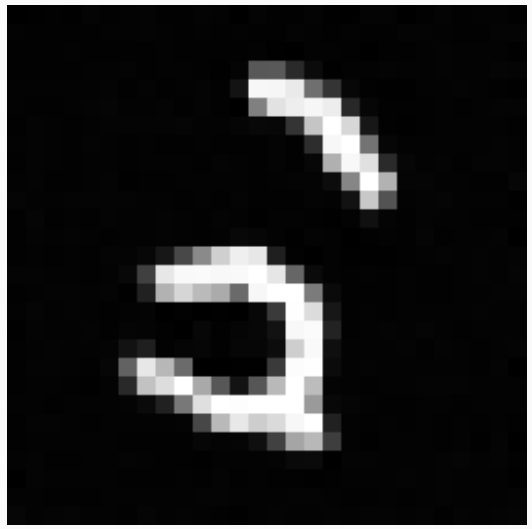
t = 800



t = 400



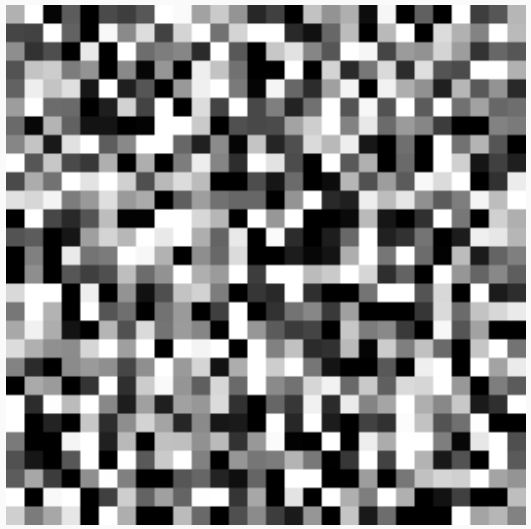
t = 50



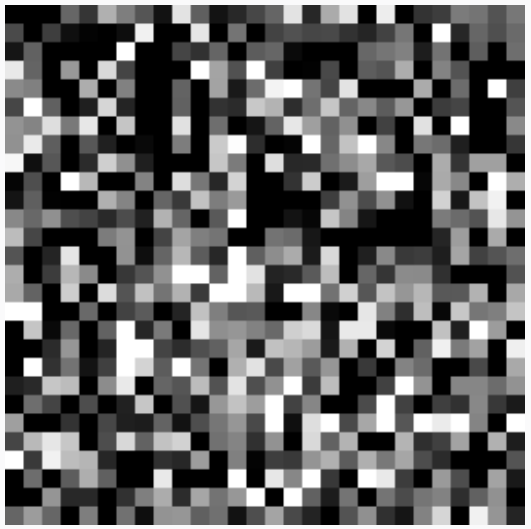
t = 0

Are all intermediate states expected to look like clean images?

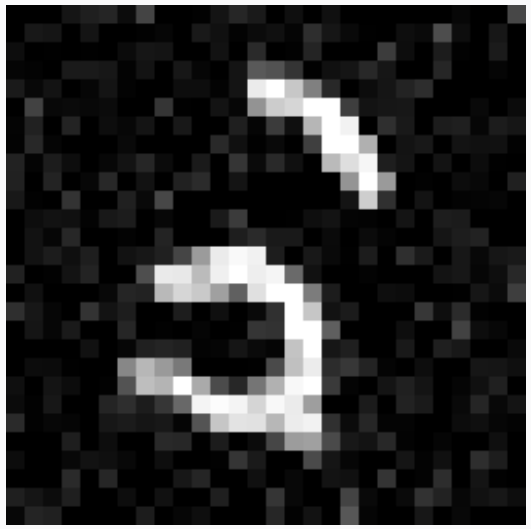
Inspect successive states on one trajectory



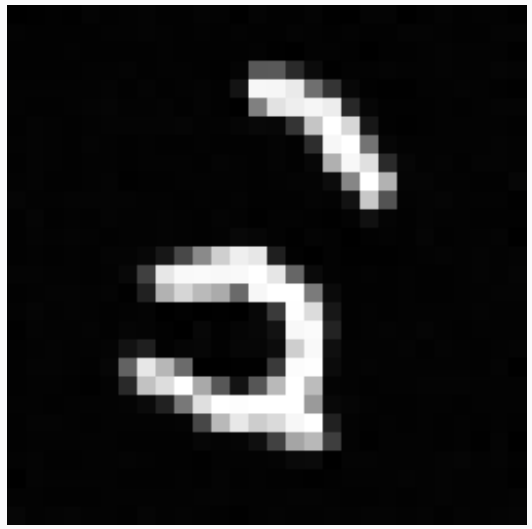
t = 800



t = 400



t = 50



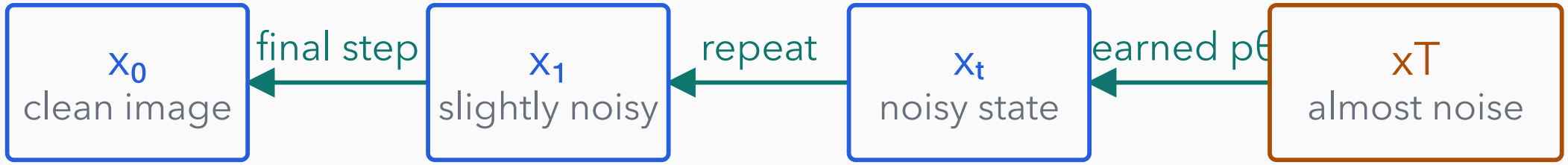
t = 0

Measured CPU replay · 1,000 steps · fixed posterior variance · no state clipping.

> No. They represent progressively less corrupted states, not independent final samples.

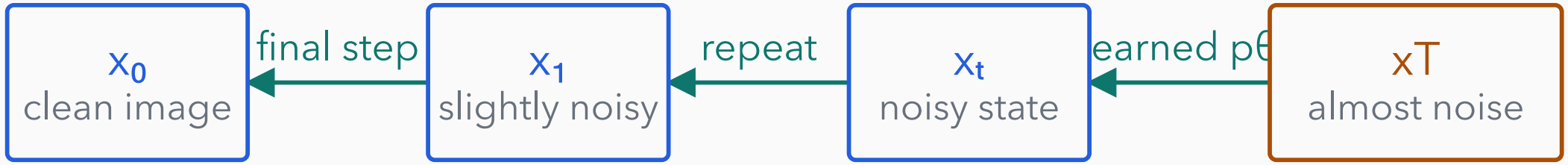
Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

The noisy states form a latent-variable model



Which states are observed in the training data?

The noisy states form a latent-variable model



$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t)$$

Only x_0. The forward process constructs auxiliary noisy states for training.

The forward process supplies a tractable training distribution

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1})$$

> Is this q trained as an amortised encoder like the VAE?

The forward process supplies a tractable training distribution

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1})$$

A variational objective compares reverse transitions with tractable training conditionals.

> Not in this DDPM: it is a fixed corruption chain.

The common noise loss simplifies the variational objective

$$L_{\text{VLB}} \supset \sum_t w_t \, \mathbb{E}\|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

The simple objective drops timestep-dependent weights; it is not numerically identical to the full negative ELBO.

> Does any noise MSE value directly equal a log likelihood?

The common noise loss simplifies the variational objective

$$L_{\text{VLB}} \supset \sum_t w_t \, \mathbb{E}\|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

$$L_{\text{simple}} = \frac{1}{CHW} \mathbb{E}_{x_0,t,\epsilon}\|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

The simple objective drops timestep-dependent weights; it is not numerically identical to the full negative ELBO.

> No. The weighting and boundary terms matter for the variational objective.

What was actually trained?

setting	saved run
data	greyscale MNIST training split
updates / batch	2,000 / 64
schedule	T = 1,000; linear β
model / objective	814,145 parameters / mean noise MSE

> Does this package claim state-of-the-art image quality?

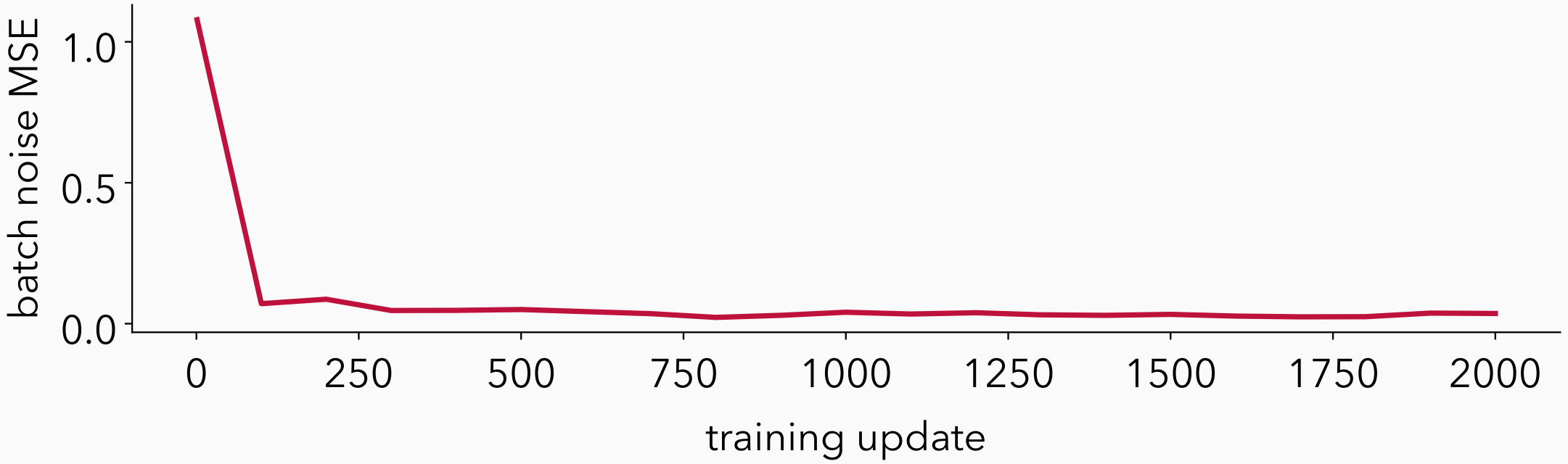
What was actually trained?

setting	saved run
data	greyscale MNIST training split
updates / batch	2,000 / 64
schedule	T = 1,000; linear β
model / objective	814,145 parameters / mean noise MSE

Saved training run: MPS, seed 667. Current checks replay this checkpoint and execute one fresh training update.

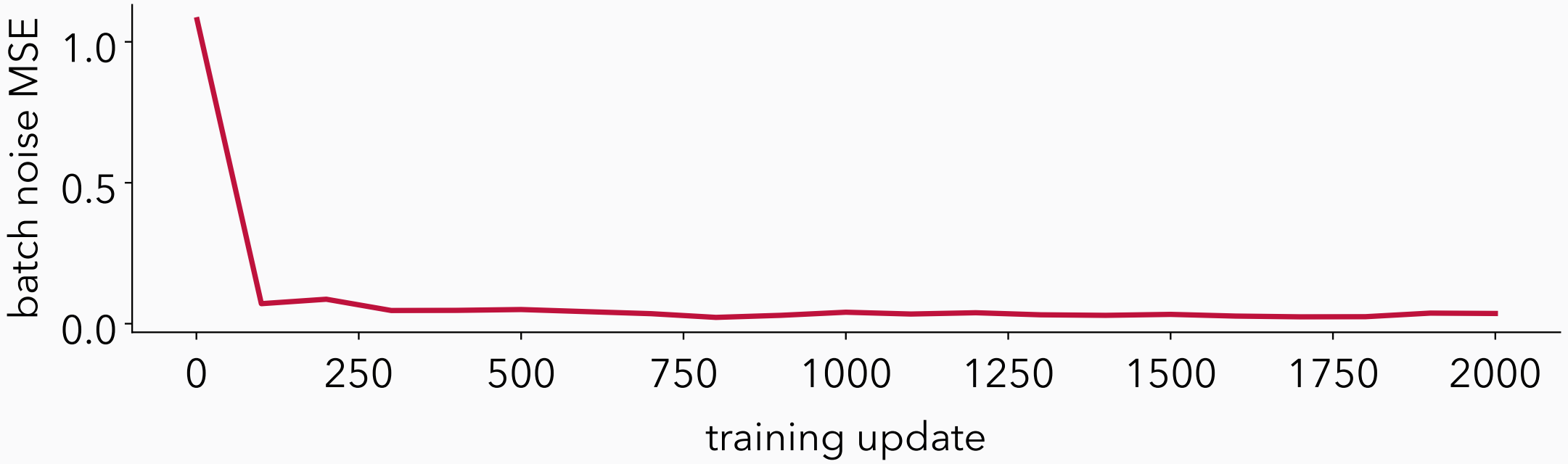
> No. It is a small teaching model with explicit training and sampling evidence.

A falling training loss is useful but incomplete evidence



> Why should successive batch values fluctuate?

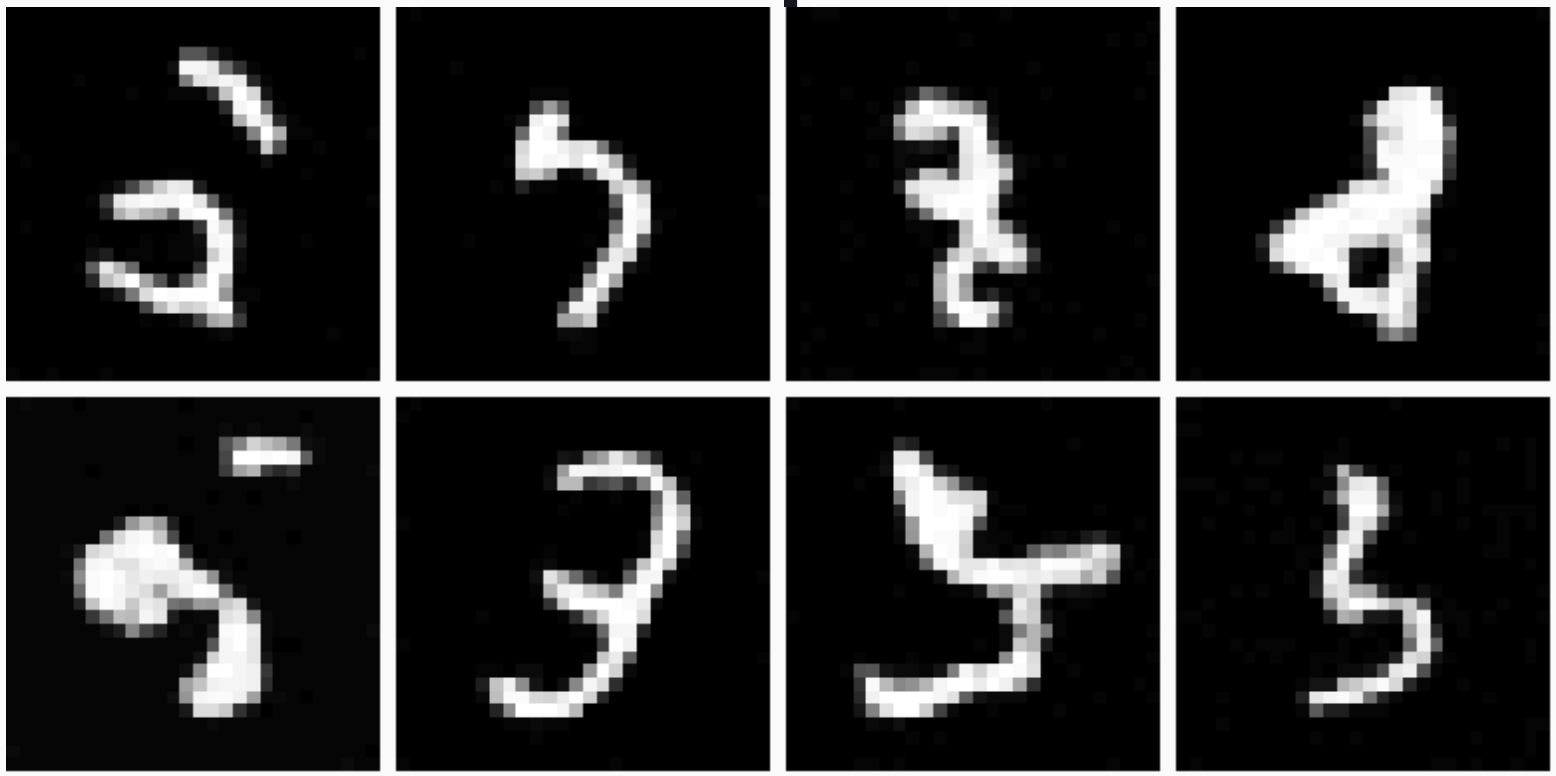
A falling training loss is useful but incomplete evidence



Each point is a recorded training batch, with new examples/t/noise; these are not matched-batch comparisons.

> They reflect different sampled training problems as well as changing parameters.

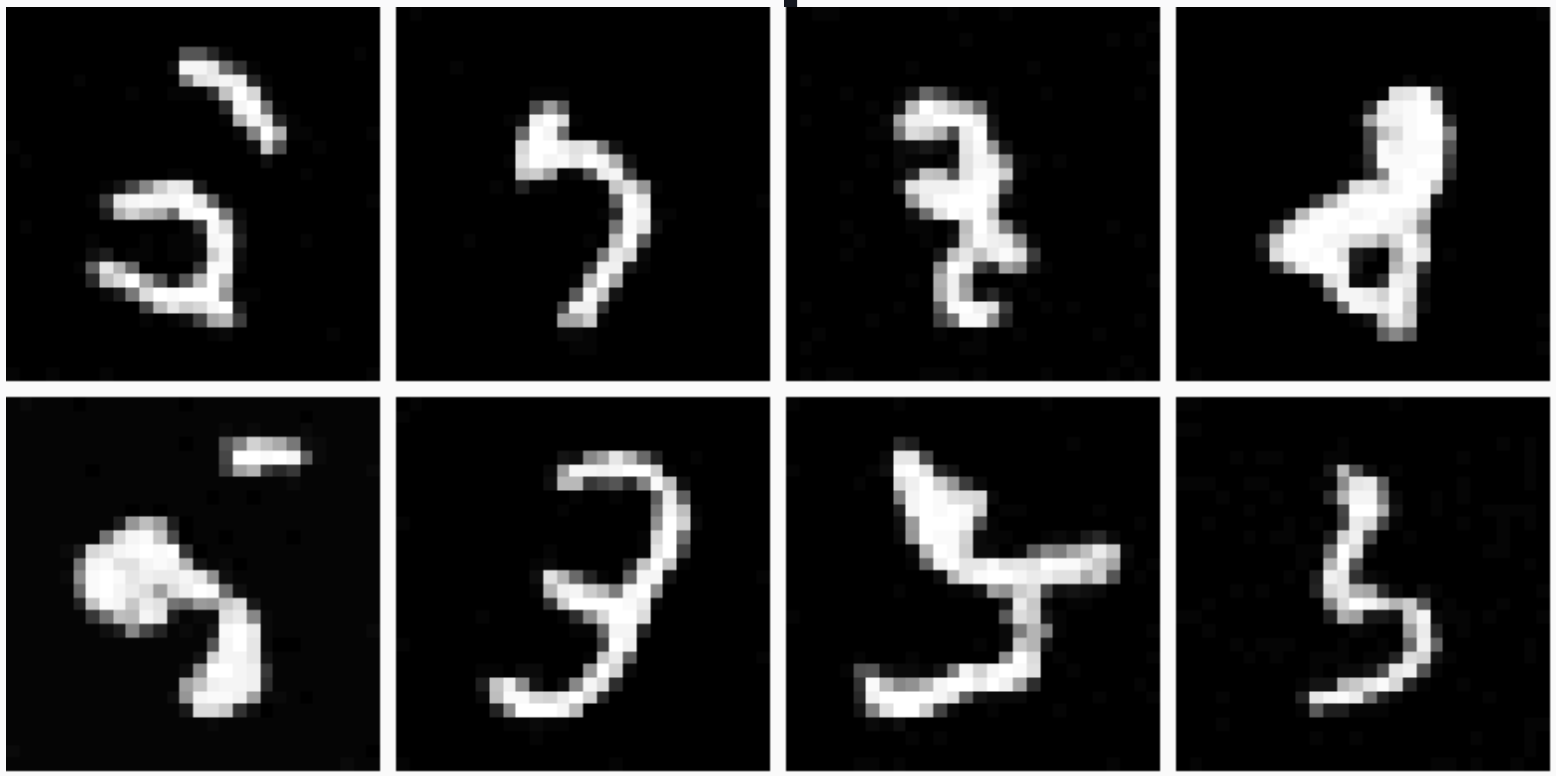
Inspect several fresh-noise samples



All 8 replay samples · seed 667 · no cherry-picked subset

What can and cannot this grid establish?

Inspect several fresh-noise samples



All 8 replay samples · seed 667 · no cherry-picked subset

Small-model samples can be malformed. A gallery is qualitative evidence, not a likelihood or coverage measurement.

> It reveals visible sample behaviour; it does not prove generalisation or distributional equality.

Live: one update, then use the saved sampler

```
check = inspect_training_update()
print(check)
samples = sample_ddpm(model, (2,1,28,28), schedule)
```

Which work is reduced in quick mode?

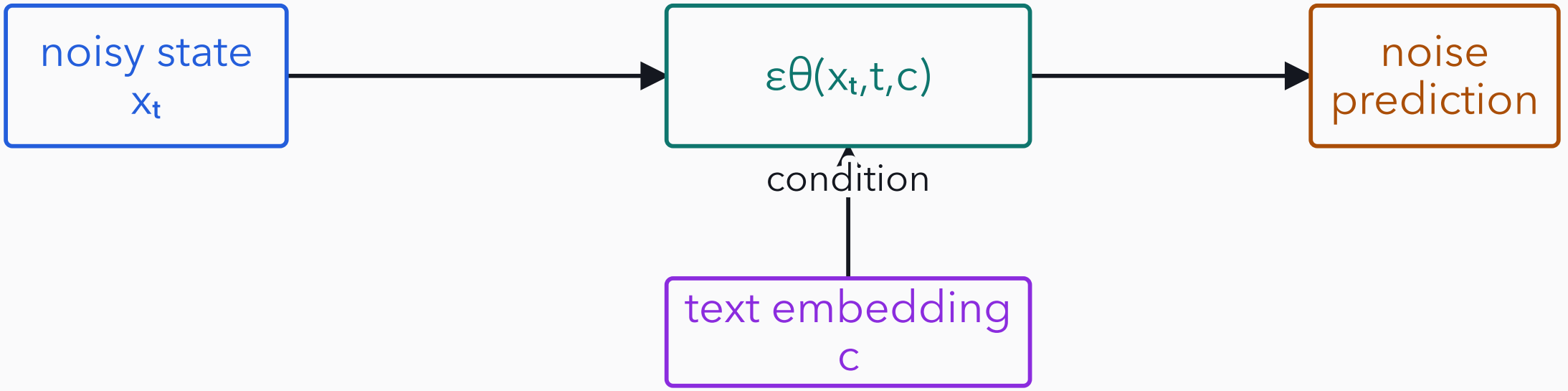
Live: one update, then use the saved sampler

```
check = inspect_training_update()
print(check)
samples = sample_ddpm(model, (2,1,28,28), schedule)
```

CPU quick mode uses a small batch and two samples. It still uses all 1,000 reverse steps.

Training iterations and number of generated samples, not the validated schedule.

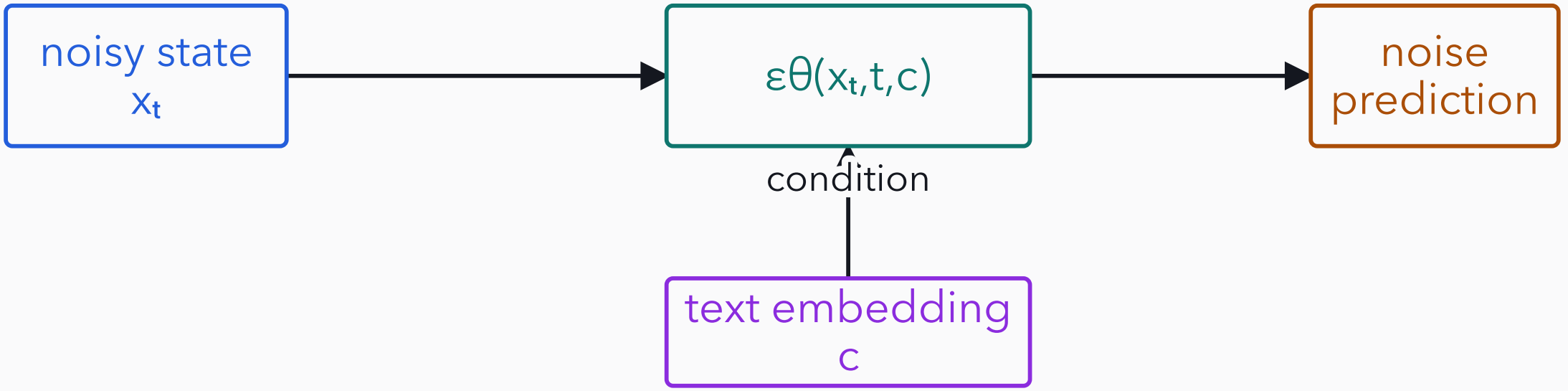
Text adds a condition to the denoiser



> Where should the text information enter the computation?

Original schematic · Rombach et al. (2022), arxiv.org/abs/2112.10752

Text adds a condition to the denoiser



The sampling loop remains recognisable; the prediction now depends on the prompt.

> Into the denoiser's prediction, rather than as a replacement for the random starting state.

Original schematic · Rombach et al. (2022), arxiv.org/abs/2112.10752

Cross-attention lets image features consult text

$$Q = h_{\text{image}} W_Q, \qquad K = cW_K, \qquad V = cW_V$$

Connection to the attention lectures; schematic conditioning block, not a full architecture survey.

> Which side asks the questions in this diagram?

Original schematic · Rombach et al. (2022), arxiv.org/abs/2112.10752

Cross-attention lets image features consult text

$$Q = h_{\text{image}} W_Q, \qquad K = cW_K, \qquad V = cW_V$$

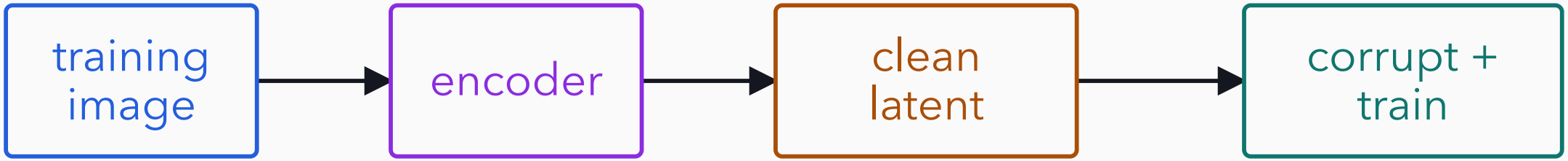
$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d})V$$

Connection to the attention lectures; schematic conditioning block, not a full architecture survey.

> The current image/latent features supply queries; text embeddings supply keys and values.

Original schematic · Rombach et al. (2022), arxiv.org/abs/2112.10752

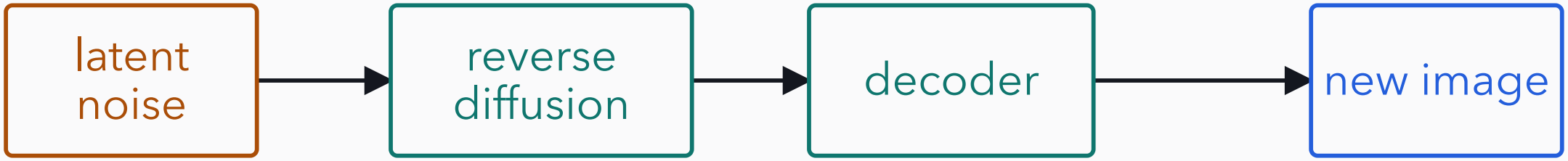
The same idea can operate in a compressed space



> Is the final image decoder the denoiser itself?

Original teaching redraw · Rombach et al. (2022), arxiv.org/abs/2112.10752

The same idea can operate in a compressed space



Generation starts from latent noise and runs the reverse model before decoding. The encoder is used to prepare training latents.

> No. Denoising occurs in latent space; a separate decoder maps the final latent to pixels.

Original teaching redraw · Rombach et al. (2022), arxiv.org/abs/2112.10752

Guidance combines two predictions

$$\hat{\epsilon} = \epsilon_\theta(x_t, t, \varnothing) + s\left[\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\right]$$

> Does larger guidance simply make every aspect of a sample better?

Original notation · Ho & Salimans (2022), arxiv.org/abs/2207.12598

Guidance combines two predictions

$$\hat{\epsilon} = \epsilon_\theta(x_t, t, \varnothing) + s\left[\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\right]$$

This convention: s = 0 is unconditional; s = 1 is conditional. Larger s extrapolates the conditional difference.

> No. It changes the sampling trade-off and can reduce diversity or create artefacts.

Original notation · Ho & Salimans (2022), arxiv.org/abs/2207.12598

Exit check: what is known during training?

We know x_0, t and the sampled ε.

> Which quantities disappear when generating from new noise?

Exit check: what is known during training?

We know x_0, t and the sampled ε.

The network receives x_t and t. The loss receives ε as its target.

> There is no clean target image or true injected ε for the current generated state.

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Exit check: reconstruct one reverse step

Predict noise → compute what → add what?

> What special action happens at the final boundary?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Exit check: reconstruct one reverse step

Predict $\varepsilon\theta \rightarrow$ compute $\mu\theta \rightarrow$ add $\sigma_t\xi$ if $t > 1$.

The clean estimate $\hat{x}_0$ is an interpretable intermediate, not the previous-state sample.

> No extra noise is added at $t = 1$.

Five lectures, five connected questions

method	question answered
VI	How do we fit an intractable posterior?
AE / VAE	How do we reconstruct, infer and sample?
GAN	Can learned feedback train a sampler?
DDPM	Can known corruption create a useful training task?

> What common habit prevents confusion across all these models?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Five lectures, five connected questions

method	question answered
VI	How do we fit an intractable posterior?
AE / VAE	How do we reconstruct, infer and sample?
GAN	Can learned feedback train a sampler?
DDPM	Can known corruption create a useful training task?

Always identify the random draw, model output, objective and actual update.

> Name what is observed, sampled, predicted, fixed and updated.

Optional: the tractable training posterior

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$$

> Why is this posterior available during training but not ordinary generation?

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Optional: the tractable training posterior

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$$

This conditional includes the clean x_0. The marginal reverse $q(x_{t-1}|x_t)$ generally does not have that information.

> Training starts from a known x_0; generation does not.

Optional: two forms of the same reverse mean

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\hat{x}_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$$

> What substitution links this to the ε-prediction formula?

Optional: two forms of the same reverse mean

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\hat{x}_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$$

$$\tilde{\mu}_t = \mu_\theta, \qquad \mu_\theta(x_{350}, 350) \approx 0.087272$$

Insert $\hat{x}_0 = (x_t - \sqrt{(1-\bar{\alpha}_t)}\varepsilon\theta)/\sqrt{\bar{\alpha}_t}$ and simplify.

Original teaching construction · Ho, Jain & Abbeel (2020), arxiv.org/abs/2006.11239

Optional: image display is not model arithmetic

A noisy state can fall outside $[-1,1]$. A screen needs a finite display range.

> Why can identical-looking grey pixels represent different model values?

Optional: image display is not model arithmetic

A noisy state can fall outside $[-1,1]$. A screen needs a finite display range.

Clip only a copy used for visualisation. Never silently feed that clipped copy back into this sampler.

> The display clips extreme values; the numerical tensor retains them.